Undergraduate Project Report
2025/26

From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol

Name:	Xiheng LI
School:	International School
Class:	2022215107
QMUL Student No.:	221166471
BUPT Student No.:	2022213192
Programme:	Telecommunications Engineering with Management

Date: 27-04-2026

Table of Contents

Abstract 3
Keywords 3
Chapter 1: Introduction 5
1.1 Problem Statement 5
1.2 Our Solution 6
1.3 Main Contributions 7
Chapter 2: Background 8
2.1 Agent Systems: Prior Work 8
2.2 What MCP Provides 8
2.3 Limits of User-space Mediation 9
2.4 Relation to Native Linux Security Mechanisms 9
2.5 Related Work 10
2.6 Threat Model 11
Chapter 3: Design and Implementation 14
3.1 Design Goals 14
3.2 System Architecture 14
3.3 Control-Plane Workflow 16
3.4 Wire Protocols 18
3.5 Manifest Semantics and Tool Identity 20
3.6 Session Management and Approval Binding 21
3.7 Kernel Module Design 23
3.8 Daemon Implementation 25
3.9 State Reconciliation and Recovery 26
3.10 Rationale for the Kernel--User-Space Split 27
Chapter 4: Results and Discussion 28
4.1 Latency and Control-Plane Overhead 28
4.2 Throughput and Scalability 34
4.3 Registry Scaling 35
4.4 Security Evaluation 36
4.5 Extended Threat Model 39
4.6 Daemon Failure and Recovery 41
4.7 Discussion 42
4.8 Threats to Validity 42

Chapter 5: Conclusion and Further Work **43**

5.1 Conclusion **43**

5.2 Reflection **44**

5.3 Further Work **45**

References **47**

Acknowledgment **51**

Appendices **52**

Disclaimer **52**

Acknowledgement of using Generative AI **53**

Project specification **54**

Early-term progress report **61**

Mid-term progress report **68**

Supervision log **75**

Additional Appendices **78**

A.1 Wire Protocol Reference 78

A.2 System Comparison Table 84

A.3 Experiment Kernel Interface 84

Risk and Environmental Impact Assessment **88**

Abstract

Modern AI agents depend heavily on external tools, yet on Linux today the operating system plays no part in gating those tool calls: the entire control plane lives in a user-space gateway. As a result, several properties that would ordinarily fall within the kernel's responsibility are left to a single daemon: ensuring that the executed tool corresponds to the approved tool, preserving the binding between requests and the sessions that opened them, and maintaining approval state across crashes.

To address these challenges, we present linux-mcp, a kernel-assisted control plane for LLM tool invocation on Linux. Semantic parsing and tool execution stay in user space; the kernel module holds only a narrow slice of control-plane state (which tools and agents are registered, which approval tickets are live, and which session a request belongs to). This design requires no modification to existing tools beyond a declarative JSON manifest and a corresponding adapter layer, and makes approval state durable across daemon failure and protected from unprivileged user-space interference.

We evaluate linux-mcp against a pure user-space baseline and a hardened seccomp variant along three axes: performance, behaviour under attack, and behaviour after daemon failure. The kernel module blocks every attempt in our targeted control-plane attack suite, against an 84.6% bypass rate for the user-space baseline, while end-to-end latency at typical payloads remains within measurement noise. An extended-attack rerun additionally identified a missing peer-credential check in the stock module, which was closed at no measurable cost. The prototype is publicly available at `https://github.com/xiheng724/linux_mcp`.

Keywords

AI Agents, Operating Systems, Tool Invocation, System Security

摘要

随着大语言模型智能体广泛通过调用外部工具执行任务，现有 Linux 操作系统缺乏对该过程的原生管控，导致控制平面的核心逻辑完全局限于用户态的网关守护进程中。这种架构使得本应由内核保障的安全与可靠性属性，过度依赖于单个用户态进程的正确性与存活性。

针对上述痛点，本文设计并实现了 linux-mcp，一种面向 Linux 的内核辅助型大语言模型智能体工具调用控制平面。该架构将语义解析与工具执行保留在用户态，而将核心控制平面状态（包含已注册的工具与智能体、生效的授权凭证及会话上下文）下沉至内核模块维护。该机制无需修改现有工具，仅需通过声明式 JSON 清单与对应适配层即可接入。借此设计，授权状态不仅能在守护进程崩溃后实现持久化，还能有效抵御非特权用户态进程的恶意篡改。

本文从性能、抗攻击与故障恢复三个维度，将 linux-mcp 与纯用户态基线及 seccomp 加固版本进行了对比评估。在针对性控制平面攻击测试集中，linux-mcp 拦截了全部攻击，纯用户态基线则有 84.6% 的攻击成功绕过，且常见负载下端到端延迟仍在测量噪声范围之内。扩展攻击实验另发现原内核模块中漏掉了一项 peer-credential 校验，已在不引入可观测延迟的前提下完成修复。原型代码已开源于 `https://github.com/xiheng724/linux_mcp`。

关键词

AI 智能体，操作系统，工具调用，系统安全

Chapter 1: Introduction

Recent advances in Large Language Models (LLMs) have enabled instruction following, multi-step reasoning, and interaction with external environments, providing the foundation for autonomous agentic systems [1, 2]. LLM-based agents are now deployed across applications ranging from direct-acting assistants to multi-step reasoning and problem-solving pipelines [3, 4], and the Model Context Protocol (MCP) has emerged as a common interface between agents and external tools [5].

1.1 Problem Statement

Consider an LLM agent asked to delete a file: a single user-space gateway reads the request, consults the tool manifest, decides whether the operation needs human approval, records the outcome, and forwards the call. If the gateway crashes between approval and execution, or if a co-located process contacts its socket at the right moment, no other authority on the host can testify whether the request was ever admitted in the first place. This is the gap between what modern agent frameworks assume about their runtime and what conventional Linux actually guarantees [6, 7].

MCP-style workflows are organised around higher-level concepts (semantic tool descriptions, session context, approval boundaries, and cross-step state) that the process, file, and socket abstractions of a conventional OS do not directly capture. When enforcement remains entirely in user space, the resulting control plane is fragile. Frameworks such as OpenClaw [8] and Hermes Agent [9] demonstrate that giving agents execution capability and persistent context opens the door to prompt injection, privilege escalation, and cross-step exploitation. Prompt injection originates at the model-input boundary and is a separate problem from control-plane integrity; this work focuses on the latter two, namely privilege escalation and cross-step exploitation, where the integrity of approval, identity binding, and cross-step state determines whether a manipulated request can take effect.

In typical deployments a single gateway process is responsible for interpreting tool manifests, binding requests to identities, and authorising execution, which effectively places the entire control plane in user space. Such a design introduces a single point of failure and expands the trusted computing base, exposing it to crashes, logic bugs, and compromised control paths [10]. Three properties are then difficult to guarantee reliably: (i) semantic consistency between declared tool behaviour and executed operations, (ii) durability of approval state across gateway crashes, and (iii) robust identity binding under replay and session confusion [4, 11].

This raises the question: which control-plane invariants in MCP-style tool invocation benefit from kernel-level enforcement, and can that enforcement improve security and resilience without making the system unacceptably slow?

1.2 Our Solution

The design of linux-mcp starts from a boundary between two classes of responsibility: control-plane properties that a user-space gateway cannot hold against its own failure, and concerns such as manifest semantics, schema validation, and tool execution where user-space flexibility remains useful. Only the first set is pushed into a Linux kernel module (admission decisions, approval-ticket state, session binding, and crash-visible audit records), while the rest is left untouched. The kernel module is restricted to checking registered identities, session bindings, approval-ticket state, and audit records that survive gateway restart; policy interpretation and tool execution remain in user space.

The architecture comprises four layers: an LLM client (`llm-app`), a manifest-aware gateway (`mcpd`), a Generic-Netlink-based kernel control plane (`kernel_mcp`), and resident tool services exposed over Unix domain sockets. Every tool invocation traverses all four layers; no request reaches a tool service without first obtaining an explicit kernel admission decision. The component responsibilities are detailed in Section 3.2, and the rationale for the kernel–user-space split is developed in Section 3.10.

This split partitions the gateway's trust along a temporal axis. At initialisation, `mcpd` is trusted to populate the kernel registries with manifest-derived tool identities and to bind sessions to peer credentials. At run time, however, it is no longer fully trusted: every `TOOL_REQUEST` is independently re-checked in the kernel, so a session-id mismatch, a skipped ticket transition, or a TOCTOU race in the daemon results in a rejection at the kernel boundary. Registry entries, approval tickets, and binding checks therefore reside in the kernel, gated by `CAP_NET_ADMIN`-protected Generic Netlink access, durable across daemon failure, and inaccessible to unprivileged processes.

Two strands of prior work frame the trade-off space. At one extreme, AIOS-style proposals argue for reimagining OS abstractions around LLM workloads [7]; this is an ambitious research direction, but it imposes a substantial deployability cost, since none of the existing Linux tooling, userbase, or security mechanisms carries over unchanged. At the other extreme, user-space frameworks such as OpenClaw and Hermes Agent preserve full deployment flexibility (they run on any Linux distribution), but, as Chapter 2 will discuss, they inherit the structural weaknesses of a monolithic user-space control plane. linux-mcp occupies the middle ground. The kernel is modified only where the gateway provably cannot defend an invariant against itself; the rest of the stack, including existing Linux security primitives such as seccomp and LSM, remains untouched and continues to operate alongside the new module. The central hypothesis arising from this placement is that binding safety and failure resilience can be improved at acceptable performance cost, a claim that Chapter 4 tests empirically.

A side-by-side comparison of linux-mcp against two representative user-space frameworks (OpenClaw and Hermes Agent) is given in Section 2.5 (Table 1).

1.3 Main Contributions

This thesis makes the following contributions:

- **A kernel-resident control plane for tool invocation.** We implement `kernel_mcp`, a Linux kernel module that maintains tool and agent registries, arbitrates invocation decisions, and manages the approval-ticket lifecycle, all behind a `CAP_NET_ADMIN`-gated Generic Netlink interface. The module contains no JSON parser and executes no tool code; its sole responsibility is to keep admission state durable across daemon failure and unreachable from unprivileged processes.
- **A manifest-aware gateway bridging semantics and enforcement.** We build `mcpd`, a user-space daemon that derives semantic hash identifiers from tool manifests, reconciles them with the kernel registry, anchors sessions to UDS peer credentials, and validates every request against its declared JSON schema before routing it through the kernel mediation path. `mcpd` concentrates all manifest-handling complexity in a single component, leaving both the kernel and the LLM client unaware of it.
- **An empirical evaluation of kernel placement as a design choice.** We compare linux-mcp against pure and hardened (seccomp) user-space baselines on latency, throughput, attack resistance, and daemon failure. The specified control-plane invariants hold under the targeted attack set, while both user-space baselines admit meaningful bypass rates; latency remains close to the user-space baseline throughout.

Chapter 2: Background

2.1 Agent Systems: Prior Work

Modern operating systems are built around the process as the primary unit of protection: address-space isolation, controlled file access, and well-defined IPC [12]. These abstractions remain fundamental. Agent-based workloads introduce a layer of intent between the user and execution that they do not directly address: an agent plans over semantically described tools, constructs structured requests, and acts on behalf of a user. Correctness therefore depends on process isolation and on whether the requested semantics match the operation that actually runs.

Several lines of work have responded to this gap. Systems such as generative agents demonstrate long-lived entities capable of maintaining memory and coordinating workflows [3]. Frameworks including LangChain [13], LangGraph [14], AutoGPT [15], CrewAI [16], and Semantic Kernel [17] structure tool composition in multi-step pipelines. More ambitious proposals embed LLM-centric services directly into a system-level kernel managing scheduling, memory, and tool access [6, 7, 18, 19]. At the application layer, OpenClaw [8] acts as a persistent user-space control layer for tool execution, while OpenHarness [20] and Hermes Agent [9] integrate control logic directly into the agent process itself; Harness [21] embeds governed AI agents in CI/CD pipelines; Apple Intelligence [22] integrates agents with on-device OS services and a private-cloud compute layer.

Section 2.3 details why this enforcement placement matters.

2.2 What MCP Provides

The Model Context Protocol (MCP), released by Anthropic in November 2024, provides a useful abstraction for structuring tool-mediated agent interaction [5]. It defines a client-host-server architecture in which tools, resources, and prompts are exposed through a standardised JSON-RPC interface. The host coordinates sessions and mediates access, while servers implement capabilities; tools are the primary mechanism by which agents take executable actions on external systems. MCP rapidly achieved broad ecosystem adoption, with implementations in Python, TypeScript, C#, and Java, and integration across major LLM providers and agent frameworks [23].

This separation simplifies interoperability and modularity. However, it also concentrates responsibility in the host: identity management, approval decisions, and consistency between tool descriptions and execution all depend on correct host behaviour. MCP defines how components communicate, but does not specify how a local system should enforce these properties. linux-mcp implements the enforcement layer that MCP leaves unspecified: it provides the local mechanism that turns the protocol's abstract trust model into a set of kernel-verified invariants.

2.3 Limits of User-space Mediation

Semantic drift. Tool invocation depends on semantically rich manifests combining natural-language descriptions, schemas, and examples [2, 5, 24]. Because both interpretation and enforcement reside in a mutable user-space process, a change in implementation or a weakened control path may result in executing a capability that no longer matches the approved semantics [25].

Identity anchoring. Agent systems bind requests to sessions or agent identifiers, not PIDs [1, 3, 15], and MCP provides no mechanism to anchor these identities in OS-visible credentials. Prompt injection and session confusion attacks can therefore cause agents to exceed their intended authority even when the gateway appears to enforce correct behaviour [11, 26, 27].

Durability. Approval decisions and pending requests may be lost on crash or restart when control-plane state is held only in memory [6]. The fragility is particularly acute in long-running agent systems where execution spans multiple steps and decisions [28, 29].

These limitations are not unique to any single framework; they reflect a broader pattern in which control-plane invariants [30] are difficult to enforce when confined to user space [31–34].

2.4 Relation to Native Linux Security Mechanisms

Any such design must coexist with existing Linux security mechanisms. Facilities such as seccomp, AppArmor, SELinux, and Landlock provide strong guarantees over system calls, resources, and process privileges [35–39]. These mechanisms operate at the level of kernel objects and access rights.

However, none of them capture higher-level concepts such as tool identity, semantic equivalence between a manifest and an execution endpoint, or the scope of an approval decision. The root cause is a layer mismatch: existing mechanisms enforce policy at the level of kernel objects (files, processes, syscalls), whereas tool identity is an application-layer concept defined by a semantic hash of a JSON manifest. Extending these mechanisms to enforce tool-identity would require either encoding tool hashes into LSM policy or hooking into the application-protocol layer, which breaks the principle that LSM hooks attach to kernel operations, not to application messages.

This gap motivates introducing a dedicated control layer that validates semantic and identity invariants at the point of tool execution, outside the mutable execution path of the user-space gateway.

Table 1: Control-plane properties of linux-mcp compared with two representative user-space tool-invocation frameworks, as configured in their default upstream documentation without external sandboxing layers. Production deployments commonly stack additional isolation on top: OpenClaw is often paired with NVIDIA NemoClaw and OpenShell sandboxing, and Hermes Agent's gateway architecture also offers Docker, SSH, Daytona, Singularity, and Modal terminal backends with container hardening. Such deployments would shift the right two columns toward stronger admission and execution boundaries; the comparison here is against the bare framework. The grey column marks the system proposed in this work.

Property	**linux-mcp (this work)**	**OpenClaw [8]**	**Hermes Agent [9]**
Control-plane placement	Split: manifest logic in user space; admission and tickets in kernel module	Standalone user-space gateway	In-process: same PID as the agent loop
Trust boundary of admission	Kernel-enforced, gated by `CAP_NET_ADMIN`	Gateway process only	Python runtime; optional Docker sandbox
Approval state	Kernel-resident tickets with explicit state machine	In-memory gateway state	Per-session module state in the agent process
Session ↔ identity binding	Hash over `SO_PEERCRED` re-checked every request	Gateway-managed session table	Session-id token; no OS-level binding
State on gateway crash	Registry, tickets, audit log persist in kernel	Lost with the gateway process	Lost with the agent process
Audit availability post-crash	Readable through sysfs without a running daemon	Application log only	Application log only

2.5 Related Work

Suppose an attacker replaces a registered tool's JSON-RPC payload at runtime, so that the `operation` field now targets a different backend handler. The socket address and the enclosing syscalls stay identical. An LSM hook at `security_socket_recvmsg` fires on the same kernel operation as before; a seccomp filter sees the same syscall numbers; a Landlock policy sees the same file descriptors. None of them has a place to hang a check that says "this tool's manifest hash no longer matches what was registered". That is the gap this section is about.

Existing kernel-native mechanisms such as seccomp-BPF, eBPF/LSM hooks, Landlock, and capability-based designs provide strong control over syscalls, kernel objects, and process-scoped authority [32, 35, 39–43]. They operate below the application-protocol layer, so they do not natively express MCP-specific concepts such as manifest-derived tool identity or semantic equivalence between a declared tool and an executed endpoint. A tool's identity in this system is a SHA-256 digest of its JSON manifest, a concept that does not exist at the syscall or VFS layer; no hook in seccomp or LSM fires at the point where a JSON-RPC payload is interpreted. Modern eBPF can perform bounded map lookups and keep per-session state in pinned maps, so neither hash checks nor ticket storage is the hard part. The harder part lies at the attachment point. eBPF LSM hooks fire on kernel operations such as socket receive and file open, not on parsed JSON-RPC messages, so any in-kernel enforcer would have to re-parse MCP fram-

ing and reconstruct tool identity from raw socket bytes on every hook invocation, all under the eBPF verifier's bounded-loop and instruction-count constraints. For this reason, these mechanisms can harden the surrounding execution environment, but they do not directly enforce the control-plane invariants targeted in this work.

The agent frameworks surveyed in Section 2.1 locate the full control plane in user space; correctness depends entirely on the in-process runtime remaining uncompromised. OpenClaw [8] operationalises this pattern at scale, and independent security analyses confirm the resulting structural vulnerabilities [26, 27]. Industrial deployments such as Harness [21] add platform-level Role-Based Access Control (RBAC) on top, but the trust anchor simply moves to a cloud control plane; it is not eliminated. Recent systems on application-layer mandatory access control including privilege-escalation defences for LLM agents [33] and programmable privilege control for tool-calling agents [34] have enforced policy inside a modified agent runtime, so invariant correctness depends on the runtime itself remaining intact. Most of these approaches keep the trust anchor in user-space or in a cloud-hosted control plane; their correctness degrades when the runtime or gateway is compromised. Apple Intelligence is a partial exception: its Private Cloud Compute design relies on Apple-controlled hardware, a hardware root of trust, and attestation mechanisms, rather than a general-purpose Linux host control plane. [22]. Table 1 contextualises linux-mcp against two representative user-space frameworks, the monolithic-gateway OpenClaw [8] and the in-process Hermes Agent [9], across the control-plane properties most relevant to this work.

AIOS-style proposals such as AIOS, OS-Copilot, MemGPT, and AgentOS [6, 7, 18, 19] pursue a broader scope: they embed LLM-specific abstractions (scheduling, memory management, tool access) into a new OS substrate. These proposals broaden scope by redesigning OS infrastructure; linux-mcp instead adds a narrow kernel role to an otherwise unchanged Linux system, targeting a bounded set of MCP-style control-plane invariants.

2.6 Threat Model

The above limitations suggest that the correctness of MCP-style systems depends on a small set of control-plane invariants, including identity binding, approval validity, and consistency between tool semantics and execution. We therefore ground our design in a threat model that focuses on attacks against these invariants in a local-host setting.

The local-adversary model is appropriate for MCP-style deployments for three reasons. First, typical MCP deployments co-locate tool services with the gateway on a single host, so the control plane itself is a local attack surface. Second, `mcpd` exposes no network interface but only a Unix domain socket, so a remote adversary without on-host code execution cannot reach the control plane at all, while one with code execution is indistinguishable from a local unprivileged process; the local model therefore captures both cases without a separate remote treat-

Table 2: Control-plane threats and defences (in scope). The last row is an observability property, not a classical attack class.

Attack class	Representative attack	Defence in linux-mcp
Spoofing	Forged `session_id` (including same-UID hijack) or unregistered `tool_hash`.	`SO_PEERCRED` binding hash over (uid, gid, pid, session_id) checked on every request; `tool_hash` must match a kernel-resident manifest entry.
Replay	Ticket reused past its TTL, or after the referenced tool was re-registered.	Tickets carry (`binding_hash`, `binding_epoch`) + TTL; re-registering a tool bumps its catalog epoch, invalidating dependent tickets and forcing session rebind.
Substitution	Manifest edited post-registration, or backend binary swapped behind a live socket.	Kernel-verified manifest SHA-256; first successful probe pins a TOFU `binary_hash` over `/proc/<pid>/exe` (composite for interpreters), re-checked on every invocation.
Escalation	Tool invoked without a valid approval via an `mcpd` logic bug (missed state transition, misrouted approval, ticket-state TOCTOU).	Kernel is the authoritative store for ticket state, so forwarding errors in `mcpd` surface as kernel-side rejections instead of silent admissions.
Audit loss	Kill `mcpd` to erase recent call history.	Per-agent `call_log` is a fixed-size kernel ring buffer, exposed read-only via sysfs; survives daemon termination.

ment. Third, two threats frequently raised in the AI agent setting fall outside this model because they live at other layers, namely prompt injection at the model's reasoning layer and supply-chain compromise of manifests at the filesystem integrity layer; neither is an appropriate target for kernel-level admission control, and both are addressed by complementary defences at those layers, as discussed later in this section. The threat model therefore focuses on what kernel enforcement can actually address: a co-located unprivileged process that can observe, forge, or replay control-plane messages exchanged with the gateway.

Concretely, the adversary model grants the attacker four capabilities: (1) arbitrary code execution under an unprivileged uid distinct from the one under which `mcpd` runs; (2) access to any Unix domain socket the gateway exposes to unprivileged processes; (3) the ability to craft and replay arbitrary JSON-RPC messages on that socket, including messages observed in prior interactions; and (4) full knowledge of the MCP protocol, tool manifests, and linux-mcp implementation. Correspondingly, the attacker interacts with linux-mcp exclusively through the `mcpd` JSON-RPC surface and holds none of the three elevated capabilities that would bypass this interface: no `CAP_NET_ADMIN`, no `ptrace` access to `mcpd`, and no write access to the manifest directory. Each boundary closes a distinct bypass path: `CAP_NET_ADMIN` would let the attacker talk to the kernel module directly via Netlink, skipping `mcpd` entirely; `ptrace` would let the attacker read or modify `mcpd`'s address space and forge control-plane state from within the gateway; and manifest write would poison the kernel registry at the next registration, since manifests are the trusted input from which tool identity is derived.

Two dominant attack categories in AI agent systems fall outside this scope. *Prompt injection*

operates at the model's reasoning layer, before any `tool:exec` request is submitted; kernel admission control has no visibility into the planning process [11, 25]. *Supply-chain compromise of manifests* requires file-system-level defences such as Integrity Measurement Architecture or device-mapper verity; linux-mcp treats on-disk manifests as trusted input and assumes the adversary lacks write access to `tool-app/manifests/`.

Trust in `mcpd` is structural, not continuous. The daemon is trusted to perform two translations correctly: at startup, converting manifest contents into SHA-256 hashes and registering them with the kernel; and at each session open, converting the client's `SO_PEERCRED` into a binding hash anchored in the kernel. Once these registrations and bindings are in place, the kernel holds canonical state and re-checks it on every `TOOL_REQUEST` and `APPROVAL_DECIDE`, so the daemon's runtime correctness is no longer load-bearing for individual request validation. This placement defends against (a) daemon logic bugs in request forwarding such as session-id mismatch, missed ticket transitions, or concurrent TOCTOU on ticket state, (b) daemon crashes, since tickets and audit state survive in the kernel, and (c) unprivileged co-located adversaries forging UDS traffic, since session binding is anchored to peer credentials they cannot spoof. It does *not* defend against a `mcpd` whose process memory is directly compromised at any point in its lifetime: a compromised daemon still holds a valid `CAP_NET_ADMIN` handle and can drive the Netlink interface with apparently legitimate calls, so runtime compromise of the gateway is explicitly out of scope and is not claimed to be contained by the kernel path. Narrowing this boundary further would require splitting the `CAP_NET_ADMIN` handle into a minimal separate service that only forwards pre-validated Netlink frames on behalf of `mcpd`, so that a memory-safety compromise of the main daemon no longer immediately grants direct kernel reach; we return to this in Section 5.3.

Within this scope, we focus on four classes of control-plane failures: (1) *spoofing* of identity or session context, (2) *replay* of previously valid requests or approval decisions, (3) *substitution* of tool identity or semantics, and (4) *escalation* through bypassing admission checks in the gateway. A fifth property, *post-compromise audit continuity*, sits alongside these four: a gateway crash should not erase the record of recent activity. Table 2 summarises the five and the corresponding kernel-anchored defences. Three of those defences go beyond manifest-hash admission alone: trust-on-first-use pinning of the backend binary identity, per-tool catalog-epoch rebind on manifest change, and a fixed-size kernel call log that survives daemon failure.

The trusted computing base is the kernel module, the operator's TOML configuration, and the manifest directory. `mcpd` is trusted to populate the kernel's view of the catalog and to bind sessions at connection time; once populated, the integrity of tickets, tool identity, and audit state does not depend on the daemon staying alive. The supplied systemd unit (Section 3.8) runs `mcpd` as an unprivileged user with only `CAP_NET_ADMIN` and `CAP_SYS_PTRACE`; this is defence-in-depth, not part of the model itself.

Chapter 3: Design and Implementation

The design question for linux-mcp is a question of placement: which parts of the control plane should the gateway not be solely responsible for holding? `mcpd` keeps everything that needs semantic flexibility; `kernel_mcp` owns the state that must survive gateway failure and stay unreachable from unprivileged processes.

3.1 Design Goals

Chapter 1.3 established that MCP-style control planes need enforcement beyond user-space self-policing. linux-mcp translates that conclusion into three design commitments.

(1) Kernel-resident admission. Every tool invocation is admitted against kernel-resident state only: the registration of the requesting agent, the presence of the target tool in the registry, and the consistency of its semantic identity with the registered manifest. The admission outcome does not depend on `mcpd`'s in-memory state at the moment of the decision.

(2) No kernel-facing obligations for tool services. Tool authors write against an ordinary user-space IPC interface; the kernel-facing protocol is confined to `mcpd` and described in Section 3.4. This preserves compatibility with existing tool ecosystems and keeps incremental deployment practical.

(3) Kernel-queryable observability. Admission, approval, and completion records are readable through kernel interfaces independently of `mcpd`'s liveness, so a daemon crash does not erase recent control-plane history.

What these commitments cover, and what they leave out (runtime semantic verification of tool behaviour, a general policy language, the broader attack surface of LLM-based systems), is fixed by the threat model of Chapter 1.3.

3.2 System Architecture

The overall architecture of linux-mcp, shown in Figure 1, comprises four runtime components.

`llm-app` is the user-facing client. It queries `mcpd` for the tool catalog, constructs invocation payloads from tool schemas, and submits requests. The client has no direct interaction with the kernel module; all control-plane operations are mediated through `mcpd`.

`mcpd` is the user-space gateway and the semantic front-end of the control plane. It is responsible for loading and validating tool manifests, computing manifest-derived semantic hashes, binding each session to the peer's `SO_PEERCRED`-derived identity and registering that binding with the kernel, exposing catalog APIs (`list_apps`, `list_tools`), communicating with the kernel over Generic Netlink, and forwarding admitted requests to tool services over Unix domain sockets.

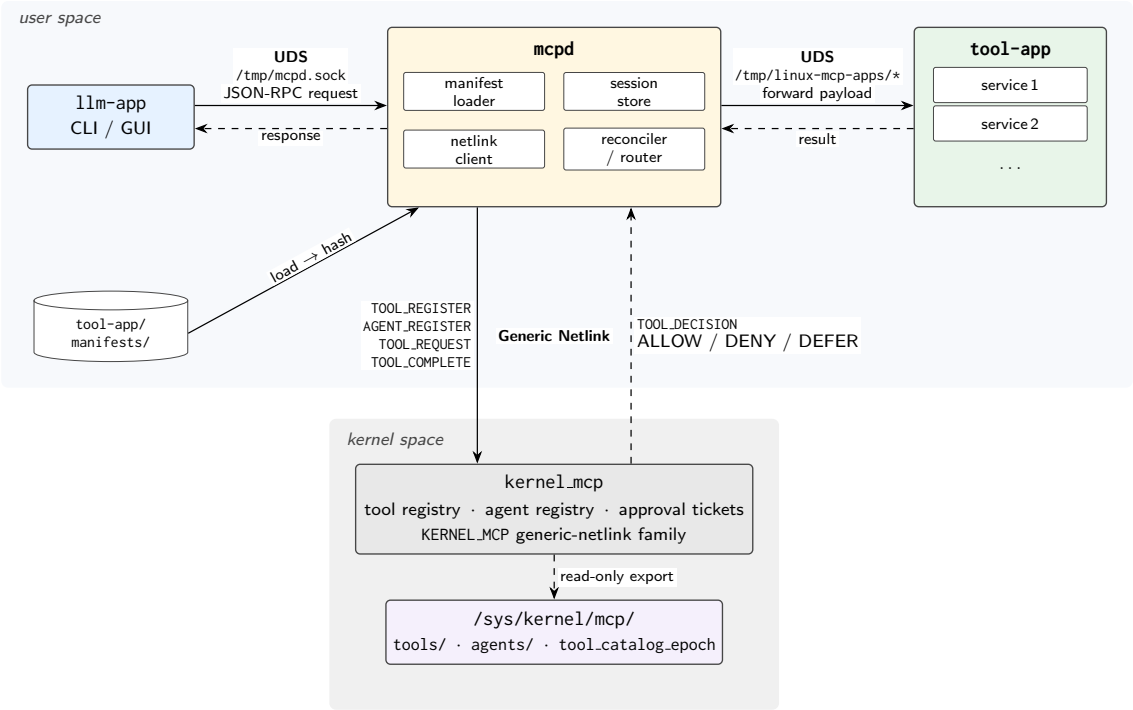


Figure 1: Layered architecture of linux-mcp, showing the four runtime components and the wire protocols connecting them.

`mcpd` is the only component that sees both tool semantics and runtime endpoints, so it is the only one that needs changing when manifests or endpoints are reconfigured.

`kernel_mcp` is the durable back-end of the control plane: a loadable kernel module implementing the `KERNEL_MCP` Generic Netlink family. It maintains the tool and agent registries, arbitrates invocation requests, manages approval ticket lifecycles, verifies the session binding hash on every request, pins a trust-on-first-use `binary_hash` for each backend process, and records per-agent invocation history in a fixed-size `call_log` ring buffer. All kernel-held state is exposed read-only through sysfs under `/sys/kernel/mcp/`, which remains readable even when `mcpd` is not running. The module does not parse manifests, execute tools, or implement a policy language; its only job is enforcing the admission-control invariants.

`tool-app` is the execution side: a collection of resident user-space services, each bound to a Unix domain socket. When a request is admitted, `mcpd` forwards the payload to the appropriate tool service, which executes in an ordinary user-space environment and returns its result to `mcpd`, which then reports completion back to the kernel. The directory `tool-app/manifests/` is the authoritative source of tool identity: `mcpd` reads manifests from here, hashes them, and registers the resulting digests with the kernel.

To clarify the separation of responsibilities across layers, Table 3 summarises each component together with its intended scope and explicitly excluded functionality.

Table 3: Concrete layers in linux-mcp.

Layer	Component	Main responsibility	Out of scope
Client layer	`llm-app` CLI/GUI	Discover tools, plan usage, open sessions, submit requests, handle approvals	Direct tool access, direct kernel communication
Gateway layer	`mcpd`	Load manifests, compute semantic hash, validate payloads, bind sessions, route requests, communicate with kernel	Durable approval and audit state that must survive daemon restart
Kernel layer	`kernel_mcp`	Maintain registries, perform admission decisions, manage approval tickets, verify session bindings, pin `binary_hash`, expose sysfs state	JSON parsing, tool execution, LLM-specific logic
Execution layer	Tool services	Execute application logic and return structured results	Any control-plane role (admission, session binding, approval tracking)

3.3 Control-Plane Workflow

The end-to-end path of a single invocation is:

$$\texttt{llm-app} \rightarrow \texttt{mcpd} \rightarrow \texttt{kernel_mcp} \rightarrow \texttt{mcpd} \rightarrow \texttt{tool-app} \rightarrow \texttt{mcpd} \rightarrow \texttt{llm-app}$$

Figure 2 illustrates this path as a sequence diagram. This path captures one `tool:exec` invocation; a one-time `open_session` handshake precedes the first such invocation on a given session and is described below. A request originates in `llm-app`, which first issues a catalog query that `mcpd` satisfies from its manifest-derived in-memory state. The kernel occupies this path not as a passive observer but as the active arbitration point: `mcpd` cannot forward a request to a tool without first obtaining an `ALLOW` decision from the kernel, and it cannot close an invocation without reporting `TOOL_COMPLETE` back through the same channel. Admission and completion are explicit, kernel-visible state transitions; the daemon no longer makes these decisions in private.

Before issuing any `tool:exec`, the client opens a session with an `open_session` call. On receiving this, `mcpd` reads the peer's credentials from the UDS connection via `SO_PEERCRED`, hashes $(\text{uid}, \text{gid}, \text{pid}, \text{session_id})$ into a binding hash, and registers the binding with the kernel. This binding is the anchor the kernel re-verifies on every subsequent request from the session.

The client uses the catalog response to construct a structured invocation payload and submits it as a `tool:exec` call to `mcpd`; the complete message schema is given in Appendix A.1.

Upon receiving the request, `mcpd` performs a series of user-space checks: it resolves the active session, validates the payload against the declared input schema, identifies the target tool, and

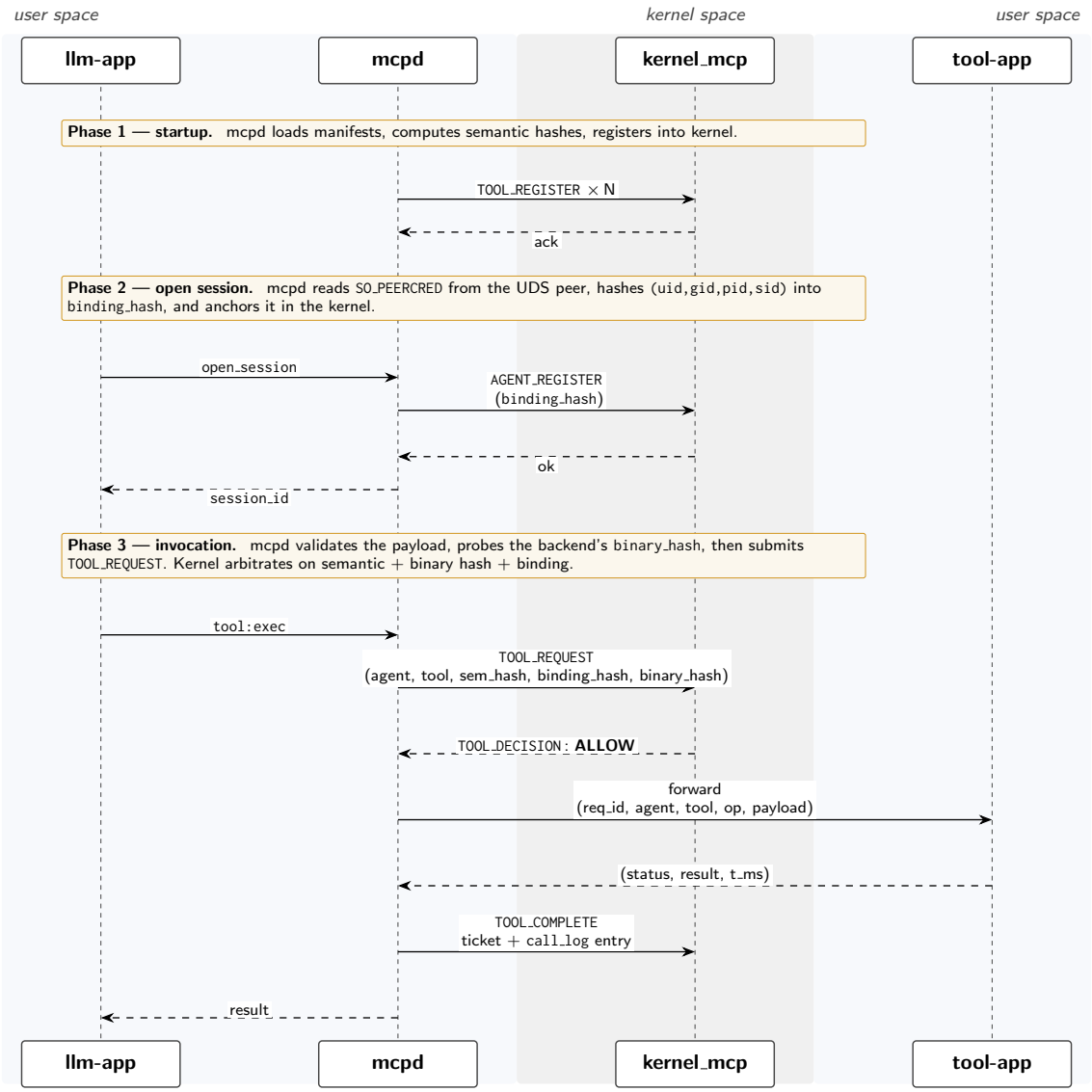


Figure 2: Invocation lifecycle in linux-mcp.

retrieves the semantic hash associated with that tool from its manifest-derived state. Only upon successful completion of these checks does `mcpd` issue a `TOOL_REQUEST` to the kernel, supplying the agent identifier, tool identifier, semantic hash, the session binding hash (derived from `SO_PEERCRED` at `open_session`), and the live-probed `binary_hash` of the backend process as parameters.

The kernel module then performs arbitration. It verifies that the requesting agent appears in the agent registry, that the target tool appears in the tool registry, that the semantic hash in the request matches the hash recorded at registration time, that the session binding hash in the request matches the one registered at `open_session`, and that the `binary_hash` (TOFU-pinned on the first non-empty value) still matches on this invocation. Based on these checks and the tool's declared policy, the kernel returns one of three decisions. `ALLOW` lets the request proceed immediately and issues an approval ticket that the subsequent `TOOL_COMPLETE` will reference.

DEFER holds the request pending an explicit APPROVAL_DECIDE, while DENY rejects it outright. The detailed semantics of these decisions are described in Figure 3.

Depending on the kernel's decision, mcpd either forwards the request to the tool service, transitions the request into a pending state awaiting explicit approval, or terminates the invocation with an error. After the tool service returns (with a normal result, an error, a timeout, or a crash caught by mcpd), mcpd issues TOOL_COMPLETE to the kernel, referencing the ticket issued at admission; the kernel records the outcome in the per-agent call_log and closes the invocation lifecycle from the control plane's perspective.

This workflow has an observability consequence: the kernel both opens and closes every invocation. An invocation that never reaches TOOL_COMPLETE, whether due to daemon failure or tool timeout, persists as an open entry in kernel state and remains inspectable via sysfs. This contrasts with a purely user-space design, in which the lifecycle of a request is only as durable as the daemon's own in-memory representation.

3.4 Wire Protocols

User-space communication in linux-mcp runs over Unix domain sockets, while the kernel control-plane path uses Generic Netlink. The default UDS variant is filesystem-path (uds_rpc). An abstract-namespace variant (uds_abstract) is also implemented and exercised end-to-end by one demo manifest, and a VSOCK variant (vsock_rpc) is reserved but not yet wired up. All UDS paths between llm-app, mcpd, and each tool service share the same framing of a 4-byte big-endian length prefix followed by a UTF-8-encoded JSON object, so a single framing routine serves both user-space endpoints. The Netlink path uses TLV-encoded attributes rather than JSON and is described at the end of this subsection.

llm-app to mcpd. llm-app sends two categories of message to mcpd: system control messages keyed by "sys" (covering list_apps, list_tools, open_session, and approval_reply) and tool execution requests keyed by "kind": "tool:exec". The two categories are distinguished by the top-level JSON key alone; tool:exec dominates in typical workloads, with sys messages appearing only at session boundaries. On deferral, the tool:exec response carries a ticket_id that the client submits with a subsequent approval_reply to unblock the request. Complete JSON schemas for all five messages are given in Appendix A.1.

mcpd to tool-app. Once the kernel returns ALLOW, mcpd forwards the request to the relevant tool service over a per-application UDS connection using a five-field request / five-field response schema. Consistent with the design goal in Section 3.1 that tool authors should not need to reason about the control plane, the tool service sees only a (req_id, agent_id, tool_id, operation, payload) request and returns (req_id, status, result, error, t_ms); it

is unaware of sessions, kernel tickets, or the Netlink protocol. The `operation` field is the dispatch string a tool service routes on and is declared per-tool in the manifest; it is distinct from the `tool_id` (numeric, used by the kernel for registry indexing) and from the tool's human-readable `name`. The full field specification is given in Appendix A.1 (Table 10).

mcpd to kernel_mcp (Generic Netlink). The kernel control-plane interface uses the `KERNEL_MCP` Generic Netlink family. Messages carry TLV-encoded attributes, and each message type is registered as a Netlink *command*; all commands except the read-only `LIST_TOOLS` dump require `CAP_NET_ADMIN`. The command set covering registration, invocation, approval, and registry observability is listed in Table 4. The attributes carrying the invariants the kernel actually checks on the critical `TOOL_REQUEST`/`TOOL_DECISION` path are shown in Table 5; the full attribute set and a reference Python framing routine are given in Appendix A.1 (Table 11).

Table 4: `KERNEL_MCP` Generic Netlink command set. All commands except `LIST_TOOLS` require `CAP_NET_ADMIN`.

Command	#	Purpose
`TOOL_REGISTER`	3	Register a tool with its semantic hash and risk flags
`AGENT_REGISTER`	9	Register an agent with its PID, UID, and binding credentials
`TOOL_REQUEST`	10	Request admission; kernel replies with `TOOL_DECISION`
`TOOL_DECISION`	11	Kernel response: `ALLOW` (1), `DENY` (2), `DEFER` (3)
`TOOL_COMPLETE`	12	Close the invocation; records execution status and latency
`APPROVAL_DECIDE`	13	Submit a user approval decision against an open ticket
`RESET_TOOLS`	14	Clear the kernel-side tool registry (issued on daemon restart)
`LIST_TOOLS`	8	Dump all registered tools (`NLM_F_DUMP` mode)

The `BINARY_HASH` attribute warrants specific mention because its production is hybrid. `mcpd` computes the digest in user space: it opens a socket to the backend, resolves the peer PID via `SO_PEERCRED`, and hashes the bytes of `/proc/<pid>/exe` at probe time. For interpreter-hosted tools whose manifest declares an entrypoint script, the digest is a composite of the interpreter executable and the on-disk script bytes, re-read on every probe, so substituting the script between invocations invalidates the pin. The kernel applies trust-on-first-use enforcement to whatever `BINARY_HASH` arrives: the first non-empty value (whether on `TOOL_REGISTER` or a subsequent `TOOL_REQUEST`) locks the slot, moving its `binary_hash_state` from `unpinned` to `live_pinned`; any later `TOOL_REQUEST` whose `BINARY_HASH` fails to match the pinned value is denied with reason `BINARY_MISMATCH`. The kernel never reads `/proc` itself; its role is to store and compare, not to probe.

Table 5: The six KERNEL_MCP attributes on the admission-decision path, each tied to one defence in the Background threat model (Table 2). Routing-level attributes (REQ_ID, TOOL_ID, AGENT_ID, APPROVAL_DECISION) and the per-tool catalog epoch are documented in Appendix A.1.

Attribute	#	Type	Description
TOOL_HASH	19	*string*	64-char hex SHA-256 of manifest fields; drives the Substitution check
BINARY_HASH	33	*string*	SHA-256 of backend's /proc/<pid>/exe (composite with script bytes for interpreter-hosted tools); drives the TOFU check
AGENT_BINDING	28	*u64*	Session binding hash over SO_PEERCRED; drives the Spoofing check
AGENT_EPOCH	29	*u64*	Session opened-at epoch; drives the cross-session replay check
DECISION	17	*u32*	Kernel verdict: 1=ALLOW, 2=DENY, 3=DEFER
TICKET_ID	22	*u64*	Approval ticket handle returned on DEFER and replayed on consume

Tool registration via manifests. Tool services do not self-register; each application is described by a JSON manifest in tool-app/manifests/ from which mcpd derives semantic hashes and issues TOOL_REGISTER Netlink commands at startup. The separation of the semantic hash (what the tool is) from the binding fingerprint over endpoint and transport (where the backend is reached), and the rebind-on-address-change mechanism it drives, are detailed in Section 3.5. The complete field specification and a representative manifest are given in Appendix A.1 (Table 12).

3.5 Manifest Semantics and Tool Identity

Defining tool identity at the right granularity is a balance between two failure modes. Binding identity too tightly to transport details such as socket paths or process identifiers means that routine operational changes (redeployments, restarts, reconfiguration) invalidate registry state even when the tool itself has not changed. Defining it too coarsely makes the system unable to distinguish a semantically modified tool from one that has merely been redeployed at a different address.

linux-mcp resolves this tension by separating semantic identity from transport location. Each tool is described by a JSON manifest, and the fields that fix its identity are tool identifier, tool name, application identifier, application name, risk tags, description, input schema, examples, path semantics, approval policy, and operation. These describe what the tool is intended to do, not where it runs. The operation field is the RPC method actually invoked on the backend, so a silent edit retargets execution to a different code path and must change the tool's identity. mcpd canonicalises these fields into a stable JSON serialisation and computes its SHA-256 digest,

represented as a 64-character hexadecimal string.

The `endpoint` and `transport` fields are excluded from the semantic hash by design. Both describe *where* a backend is reached, not *what* it does, and routine operational moves (switching a service to a new socket path, or between transport variants such as `uds_rpc` and `uds_abstract`) should not cause a tool to look semantically different to planners or to kernel-side hash checks. `mcpd` therefore tracks these two fields in a separate *binding fingerprint*, also a SHA-256 digest, maintained alongside but distinct from the semantic hash. A change in either field produces a new binding fingerprint and drives an unregister-then-re-register cycle against the kernel. The cycle clears the kernel's trust-on-first-use `binary_hash` slot on the grounds that the backend reached at the new address may not be the same process. The semantic hash is unaffected by the move, so planners and approval records that reference the tool remain valid.

Any modification to a semantically significant field, including the input schema, risk tags, description, approval policy, path semantics, or `operation`, produces a different semantic digest, which the kernel detects as a mismatch on the next `TOOL_REQUEST`. The semantic hash is the single artefact both sides check, computed in user space from the manifest and verified by the kernel.

3.6 Session Management and Approval Binding

Approval would be straightforward if defer and execute were atomic, but they are not: the request that returns after the user decision has to prove it is the same request, in the same session, against the same tool, or a TOCTOU window [44, 45] opens between the two events. Each session is bound to the real peer credentials of its Unix domain socket connection [46]. `mcpd` hashes `(uid, gid, pid, session_id)`, where `session_id` is per-session random, through BLAKE2b to an 8-byte output and stores the 64-bit digest as the binding hash alongside a session epoch. Including `pid` and a random `session_id` stops two processes sharing a UID from reusing each other's session. The width matches the `AGENT_BINDING` Netlink attribute, so `mcpd` resubmits the hash on every `TOOL_REQUEST` and the kernel compares it against what was registered at `open_session`. A `DEFER` response creates an approval ticket tagged with this binding context and a five-minute expiry (overridable per request), and the kernel re-checks the binding when `APPROVAL_DECIDE` arrives. Session-scoped binding plus per-request re-verification rules out replay across sessions: the state at deferral is the state at execution.

Catalog-epoch rebind on manifest change. A second form of staleness arises when the catalog changes while a session is open: without further mechanism, a client could keep invoking a re-registered tool under its pre-change binding. Each tool records a `registered_at_epoch` at (re-)registration, each session records an `opened_at_epoch` at open time, and a `TOOL_REQUEST` is denied with `catalog_stale_rebind_required` when the targeted tool's `registered_at_`

epoch exceeds the session's opened_at_epoch. Because the check is per tool, only the affected tool's re-registration invalidates a pending session; editing an unrelated manifest advances the shared tool_catalog_epoch counter without disturbing anything else.

Beyond the kernel ticket flow, linux-mcp supports two independent approval mechanisms: one in the kernel and one in the client, driven by different manifest fields and enforced at different layers.

(1) Kernel-mediated approval (DEFER ticket flow). The kernel decides between ALLOW, DENY, and DEFER on every TOOL_REQUEST based on the tool's risk_flags, a bitmask derived from the manifest's risk_tags at registration time. Tools tagged FILESYSTEM_DELETE, DEVICE_CONTROL, EXTERNAL_NETWORK, PRIVILEGED, or IRREVERSIBLE are deferred by default. The five validity checks that distinguish ALLOW from DENY, and the state machine that governs the resulting tickets, are given in full in the Kernel Module Design section. When a request is deferred, mcpd records the pending operation and returns the ticket identifier to the client. After the user approves, mcpd first sends APPROVAL_DECIDE to the kernel to transition the ticket, then replays the original request with the ticket identifier. The kernel consumes the ticket during the second TOOL_REQUEST and returns ALLOW.

(2) Client-side local approval (payload-aware prompt). llm-app consults each tool's approval_policy field before sending tool:exec to mcpd. Most tools omit the field and incur no local prompt. A tool may declare approval_policy.user_confirmation with a condition such as when = "path_outside_repo", in which case the client resolves the payload's declared path fields against the repository root and prompts the user if any of them resolve outside. The demo tool write_host_text_file in 04_desktop_app.json uses exactly this to require confirmation before writing to the host filesystem outside the project directory. A declined local prompt terminates the request in user space. No tool:exec is sent to mcpd and no kernel ticket is created.

Relationship between the two mechanisms. The two mechanisms gate on different things, not the same checks. Kernel DEFER is *payload-agnostic*: every invocation of a high-risk-flagged tool is gated regardless of arguments. The local prompt is *payload-aware*: only requests whose arguments meet the declared condition are gated. A single tool can trigger both, in which case execution proceeds only after the local prompt is approved, an APPROVAL_DECIDE succeeds, and the ticket-bearing second TOOL_REQUEST is admitted.

Figure 3 illustrates the kernel-side state machine. Approval always resolves *before* tool execution, so no deferred request bypasses the gate and every deferred execution consumes an explicit ticket. The kernel verifies only what it needs to: binding context preserved, ticket valid and unconsumed, and the DEFER → ALLOW transition explicitly authorised [47].

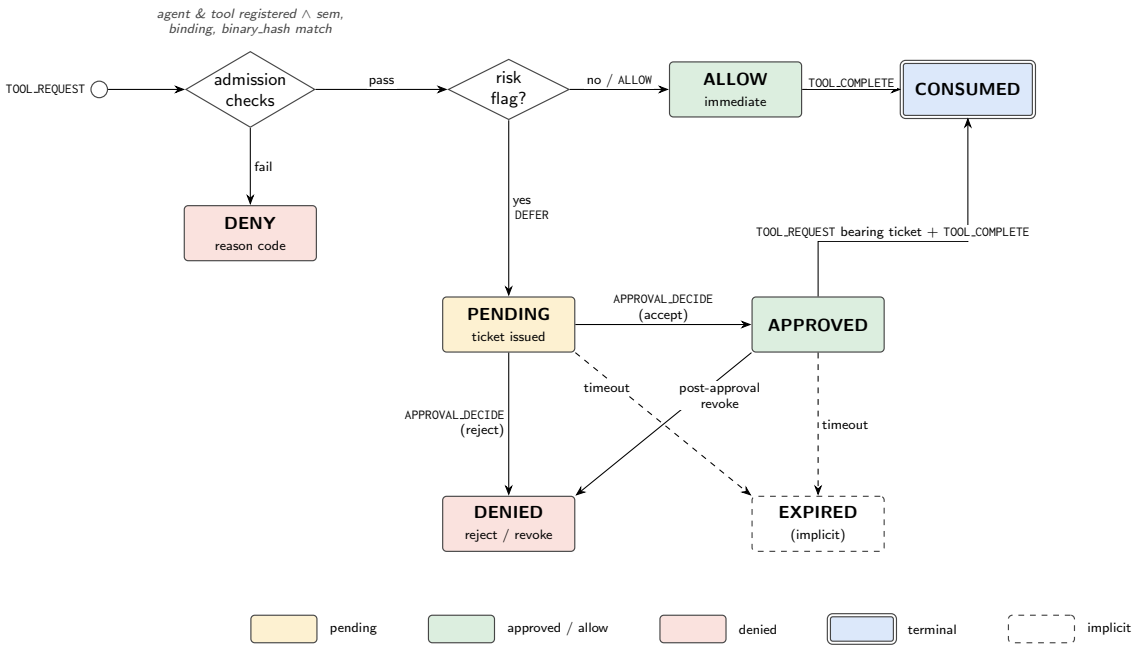


Figure 3: Approval-ticket flow in linux-mcp: the ALLOW/DENY/DEFER state machine enforced by the kernel module.

3.7 Kernel Module Design

The kernel module `kernel_mcp` is loadable and exposes the `KERNEL_MCP` Generic Netlink family, with eight commands that cover registration, invocation, approval, and registry observability (`TOOL_REGISTER`, `AGENT_REGISTER`, `TOOL_REQUEST`, `TOOL_DECISION`, `TOOL_COMPLETE`, `APPROVAL_DECIDE`, `RESET_TOOLS`, and `LIST_TOOLS`). Generic Netlink was chosen over `ioctl` or a character device because it provides structured TLV attributes with kernel-side schema validation, native support for dump-style enumeration (used by `LIST_TOOLS`), and per-command `GENL_ADMIN_PERM` gating that integrates cleanly with Linux capabilities [48].

The module's tool registry maps tool identifiers to their manifest-derived semantic hashes and associated metadata, with each entry also carrying risk flags, a per-tool `registered_at_epoch`, and the TOFU `binary_hash` slot. A separate agent registry records agents permitted to submit requests through the control plane, together with the `binding_hash`, `binding_epoch`, and `opened_at_epoch` captured at `open_session`. Approval tickets are heap-allocated kernel objects with an explicit state machine. The normal path runs `PENDING` → `APPROVED` → `CONSUMED`. A reject takes `PENDING` to `DENIED`. A post-approval revoke takes `APPROVED` to `DENIED`. `EXPIRED` is implicit; an expired ticket is removed from the table, not marked with a stored state.

The kernel admits a `TOOL_REQUEST` only when five conditions hold simultaneously, which keeps the arbitration logic narrow. The submitting agent must appear in the agent registry, the target tool must appear in the tool registry, the semantic hash supplied by `mcpd` must match the hash recorded at registration, and the session `binding_hash` must match the one captured at

open_session. If the TOFU slot is already pinned, the request's binary_hash must match the pinned value as well. Any failing check produces DENY with a specific reason code. Otherwise the kernel consults the tool's risk_flags, and a tool carrying FILESYSTEM_DELETE, DEVICE_CONTROL, EXTERNAL_NETWORK, PRIVILEGED, or IRREVERSIBLE is deferred and receives a fresh approval ticket, while everything else gets ALLOW.

Trust-on-first-use backend binary pinning. Semantic-hash admission guards what a tool claims to be, not which binary actually serves the socket. The kernel therefore keeps a per-tool binary_hash slot that mcpd fills on a live probe. mcpd reads the backend's peer PID via SO_PEERCRED and hashes /proc/<pid>/exe. For interpreter-hosted tools the digest combines the interpreter with the on-disk script, so a script swap is still caught. The slot locks on the first non-empty digest, which usually arrives on TOOL_REGISTER but may arrive on the first successful TOOL_REQUEST if the backend comes up later. Any subsequent TOOL_REQUEST whose digest does not match is rejected with reason BINARY_MISMATCH. A binary_hash_state field distinguishes unpinned from live_pinned, since both previously read as an empty string.

Kernel-resident call log. Each agent carries a fixed-size 32-slot circular buffer of invocation records. A record is a flat struct holding a monotonic sequence number, a nanosecond timestamp, the request and tool ids, a status (one of OK, ERR, DENY, DEFER), execution latency, a tool status code similar to a process exit code, 8-byte payload and response hash prefixes, and up to 48 leading bytes of any error output. Completed invocations log from cmd_tool_complete; denied or deferred ones log from cmd_tool_request, so nothing escapes the audit buffer. The kernel parses no JSON. mcpd precomputes the hashes and passes them as fixed-width attributes. The buffer is exposed through sysfs as a binary attribute and stays readable after a mcpd crash. This provides crash-visible audit, not tamper-evident provenance: the buffer is bounded and the hashes are 8-byte prefixes.

Sysfs surface. Each tool directory under /sys/kernel/mcp/tools/ exposes name, semantic hash, binary_hash, binary_hash_state, and registered_at_epoch. The global tool_catalog_epoch sits at the top of the tree. Per-agent entries under /sys/kernel/mcp/agents/ expose per-state counters (allow_count, deny_count, defer_count, completed_ok_count, completed_err_count, last_exec_ms), a human-readable last_reason, and the call_log above with its head and count indices. An operator can read what the kernel regards as authoritative without relying on daemon-side logs.

3.8 Daemon Implementation

Most runtime work happens in mcpd; the kernel module is consulted only on admission decisions. The daemon splits across four files: manifest_loader.py (manifest parsing and semantic hashing), session_store.py (session bindings), netlink_client.py (kernel Netlink traffic), and server.py (JSON-RPC dispatcher and backend probe).

Kernel state reconciliation. The manifest directory is the source of truth, but the kernel registry survives across mcpd restarts. The reload path has to keep the two in sync without invalidating existing sessions unnecessarily. On each reload mcpd diffs the previous and new registries per tool and picks one of five reconcile actions. KEEP applies when both fingerprints match, so no kernel call is issued. ADD and REMOVE handle tools that appear or disappear. RE_REGISTER_SEMANTIC covers a changed semantic hash on an unchanged backend; the in-place re-register preserves the kernel's TOFU binary_hash slot. RE_REGISTER_BINDING_MOVE fires when transport or endpoint changed: the backend may now be a different process, so mcpd unregisters first to clear the TOFU slot, then re-registers so the next probe pins a fresh digest.

Probe-failure fail-closed. The TOFU binary_hash mechanism only works if mcpd never quietly substitutes an empty digest for a failed probe. When mcpd cannot reach the backend, cannot read /proc/<pid>/exe, or does not recognise the transport, the probe returns a non-live result and the exec path refuses the invocation. No empty BINARY_HASH flows upward. A failed probe will not reset a pinned slot and let a swapped backend start clean, and a cold registration before the backend comes up will not install a blank pin a later swap could inherit. Audit records for probe refusals carry source=probe, distinct from kernel denials (source=arb) and client declines (source=user_decline).

Netlink client. mcpd keeps one long-lived Netlink socket to kernel_mcp rather than reopening one per request. Socket setup is measurable on a microsecond-scale path, and a persistent connection simplifies ordering APPROVAL_DECIDE against its eventual TOOL_REQUEST replay. Optional per-stage latency counters (session lookup, kernel arbitration, tool execution, end-to-end) plug into this path for the overhead analysis in Chapter 3.10.

Deployment and fail-closed configuration. mcpd needs CAP_NET_ADMIN for Netlink administration and CAP_SYS_PTRACE for reading /proc/<pid>/exe across uid boundaries during the backend probe. The supplied systemd unit runs mcpd as a dedicated unprivileged user, grants both caps via AmbientCapabilities, and bounds them in CapabilityBoundingSet so an execve'd child cannot widen the cap set. Two further directives close gaps the threat model would otherwise leave open: NoNewPrivileges=true blocks SUID-based privilege es-

calation in any child, and `MemoryDenyWriteExecute=true` rejects writable executable pages and so blunts a memory-corruption exploit class. The remaining systemd hardening (`ProtectSystem=strict` with a `/tmp` allowlist, `ProtectKernelTunables`, `ProtectKernelModules`, `ProtectControlGroups`, `RestrictSUIDSGID`, `LockPersonality`, `PrivateDevices`) shrinks the blast radius of a compromised daemon at no operational cost.

A separate question is which UIDs may serve as tool backends. `mcpd` checks `SO_PEERCRED` against `allowed_backend_uids` on every probe and exec dial. Defaulting to `{0}` (root only) rejects every realistic non-root backend and leaves `binary_hash` unpinned; defaulting to `$SUDO_UID` folds the launcher script's correctness into the daemon's trust boundary. `mcpd` therefore refuses to start without an explicit TOML list or an explicit `LINUX_MCP_TRUST_SUDO_UID` opt-in; backend UIDs come from configuration, never from the launch environment.

3.9 State Reconciliation and Recovery

linux-mcp treats a daemon restart as a recovery problem, not a process-management detail. The three layers of state it keeps sit on different sides of the crash boundary. The kernel holds the tool registry, approval tickets, the `tool_catalog_epoch` counter, and each agent's `call_log`. All of that survives a `mcpd` crash. The manifest set on disk is the authoritative source of tool identity and is unaffected by restart. Open sessions, peer-credential bindings, and pending request maps live only in `mcpd`'s memory and are discarded when the process exits.

The cold-start procedure follows directly from this partition. At startup, `mcpd` issues `RESET_TOOLS` to clear the kernel tool registry and then re-registers each manifest tool unconditionally, which guarantees the kernel and the on-disk manifest set agree before any client is accepted. `RESET_TOOLS` alone does not bump the catalog epoch; the per-tool register calls that follow do. A surviving session holding a pre-restart `opened_at_epoch` will therefore receive `catalog_stale_rebind_required` on its next request against any re-registered tool. Cold start deliberately does not attempt diff-based reconciliation against the surviving kernel registry: the per-tool diff path described in Section 3.8 applies only to live manifest reloads while `mcpd` is running. Optimising cold start to reuse residual kernel state under an explicit divergence signal is left to future work (Section 5.3).

This split preserves audit and admission state without pretending that a crashed daemon can safely continue pre-existing sessions. Approval tickets and registry history stay inspectable through sysfs across the restart, so an operator can still read what recent activity occurred while `mcpd` was down or unresponsive. Any surviving ticket whose referenced session no longer has a live `mcpd`-side binding simply expires at its TTL. Clients reopen any session that pre-dates the restart on their first post-restart request. The client library handles this rebind automatically when it sees the `catalog_stale_rebind_required` reason.

A kernel module reload sits outside this recovery model. Unloading `kernel_mcp` drops all

kernel-held state (registry, tickets, epochs, call logs), so the operator must treat the next start as a fresh system. `mcpd` re-registers from manifests as usual, but any historical audit kept only in the now-gone kernel buffer is lost.

3.10 Rationale for the Kernel–User-Space Split

Section 2.5 established the main layer mismatch: MCP admission decisions depend on parsed request state and manifest-derived tool identity, while eBPF/LSM hooks observe kernel objects and syscall events. In implementation terms, this ruled out an eBPF-only design because it would have required reconstructing MCP framing and request identity from raw socket bytes under verifier constraints. Direct LSM hooks had the same attachment-point problem: no hook naturally carries both the request body and the registered tool identity.

A capability-style design was also considered, but it would have changed the programming model for tool services. Each backend would need to obtain, store, and present kernel-facing capabilities during invocation. That conflicts with the design goal that tool authors only provide a JSON manifest and continue to speak ordinary user-space RPC.

Generic Netlink was chosen because it gives `mcpd` a structured kernel interface for exactly the state transitions this design needs: tool registration, agent registration, request arbitration, approval decisions, completion reporting, and registry dumps. It keeps JSON parsing, schema validation, and tool execution in user space, while letting the kernel hold the small amount of durable state needed for admission. The cost is a loadable kernel module rather than a policy hook or BPF program, but the module remains limited to registry membership, ticket state, session binding, and audit records.

Chapter 4: Results and Discussion

linux-mcp is evaluated along four dimensions: control-plane latency and its breakdown across invocation stages, throughput under concurrent load, resistance to a representative set of control-plane attacks, and system behaviour under daemon failure and restart. The central claim under test is that relocating admission-control state into the kernel strengthens enforcement without making the tool invocation path prohibitively expensive. Two baselines isolate this effect from user-space hardening alone: a conventional user-space mediator (**user-space**), in which registration, admission, and approval all reside in the daemon, and a hardened variant (**seccomp**) that augments the same control path with stricter syscall filtering, additional validation, and audit logging.

The prototype under test comprises 43 tools across 14 applications; all three systems operate over this same tool deployment throughout the evaluation. All experiments were conducted on a VMware guest (Apple aarch64 host, 4 vCPU, 8 GB RAM) in a headless configuration with no concurrent user-space workloads. Latency results are reported over $n = 30$ independent runs of 2000 requests each, giving a standard error of the mean small enough to support Welch's t-test comparisons at $\alpha = 0.05$ while remaining within the hypervisor timing noise of a virtualised guest. Throughput and the category-level security evaluation are drawn from the same run, using the harness's built-in concurrency sweep and ten repeats of each attack case. All reported p-values are two-sided Welch's t-tests [49] comparing per-repetition mean latency against the `user-space` baseline.

4.1 Latency and Control-Plane Overhead

Table 6 and panel (a) of Figure 4 report mean end-to-end latency for each system at three payload sizes. The 100 B and 10 KiB points correspond to typical MCP tool invocations (short JSON commands and small structured responses); the 1 MiB point is a contrived stress case, used solely to probe how the control plane behaves when the payload dominates the cost of a request, and is not intended to reflect everyday tool traffic. At 100 B, the kernel path and the user-space

Table 6: Mean end-to-end latency by payload ($n = 30$ runs $\times$ 2000 requests). Δ columns report the relative change against the `user-space` baseline on the same row; p-values are two-sided Welch's t-tests against that baseline, $^{\text{ns}} = p > 0.05$. Bold marks the row minimum.

	user-space	**seccomp**			**kernel**		
Payload	**ms**	**ms**	Δ	p	**ms**	Δ	p
100 B	**0.854**	0.867	+1.5%	0.48^{ns}	**0.854**	0.0%	0.98^{ns}
10 KiB	**0.863**	0.922	+6.8%	2.1×10^{-4}	0.899	+4.2%	0.10^{ns}
1 MiB	7.748	9.843	+27.0%	$< 10^{-4}$	**7.487**	−3.4%	2.7×10^{-4}

baseline are within measurement noise: both at 0.854 ms ($p = 0.984$). At 10 KiB the kernel path is nominally slower (0.899 ms versus 0.863 ms, +4.2%), but the difference does not reach significance ($p = 0.097$), a sub-millisecond effect lost in the timing noise of a VMware guest at $n = 30$.

The 1 MiB row provides the only statistically clean comparison in the table, with the kernel path at 7.487 ms against 7.748 ms for plain user-space (a 3.4% reduction, $p = 0.00027$) and 9.843 ms for the hardened variant. This is counter-intuitive: adding a privilege boundary is usually expected to cost latency, not return it. Panel (c) of Figure 4 shows the same data as a ratio against the plain `user-space` baseline, with the y-axis zoomed around the parity line so that the sub-4% effects at 100 B and 10 KiB are visible in the same frame as the larger effect at 1 MiB; read that way, `seccomp` sits above parity at every payload size while the kernel path crosses below parity only at 1 MiB. The seccomp penalty at large payloads has a clean explanation: BPF per-syscall filtering pays a constant cost on every `sendmsg`/`recvmsg` crossing, and a megabyte payload forces many more such crossings through the tool-forwarding socket than the smaller payloads, a cost the kernel path does not incur. The 3.4% speedup of the kernel path over plain user-space at 1 MiB is harder to attribute confidently from the available stage-level instrumentation: both paths admitted these requests (DENY would produce a different latency distribution) and both subsequently copied the same 1 MiB payload through the same `mcpd` → tool-app UDS, so an "avoided copy" explanation does not hold. Plausible contributors include scheduler effects (`mcpd` releases the CPU while waiting on the kernel Netlink reply, which may interact favourably with how the guest kernel schedules a 1 MiB `sendmsg`) and hypervisor-level timing jitter (a 3.4% effect at the 7–8 ms scale falls within the noise envelope a VMware guest typically exhibits at this latency range); the present instrumentation does not separate them. We report the effect because it is statistically clean ($p = 2.7 \times 10^{-4}$ at $n = 30$) but treat it as evidence that the kernel path does not impose a measurable overhead at this payload size, not as evidence of a positive speedup mechanism.

The p_{95} picture, shown in panel (b) of Figure 4, matches the means for the kernel path: at both 100 B and 10 KiB the kernel p_{95} is statistically indistinguishable from user-space ($p = 0.51$ and $p = 0.16$). The seccomp p_{95} already separates from user-space at 10 KiB (1.078 ms vs 1.003 ms; $p = 6.4 \times 10^{-3}$), so the seccomp tail is the first to widen under load even though the means are still close. At 1 MiB the kernel p_{95} is 8.422 ms, against 8.716 ms for user-space ($p = 0.049$) and 10.903 ms for seccomp ($p < 10^{-4}$). The kernel tail does not widen relative to user-space across any of the measured payloads.

The stage-level breakdown in panel (d) of Figure 4 confirms that kernel arbitration is a minor contributor at all payload sizes. Mean arbitration time is 0.028 ms at 100 B and 10 KiB, and 0.038 ms at 1 MiB. At 1 MiB, tool execution accounts for 89.3% of the instrumented stage total (session lookup + arbitration + tool execution = 4.351 ms on the kernel path) and 89.9% of the same total on the user-space path; arbitration contributes less than four hundredths of a

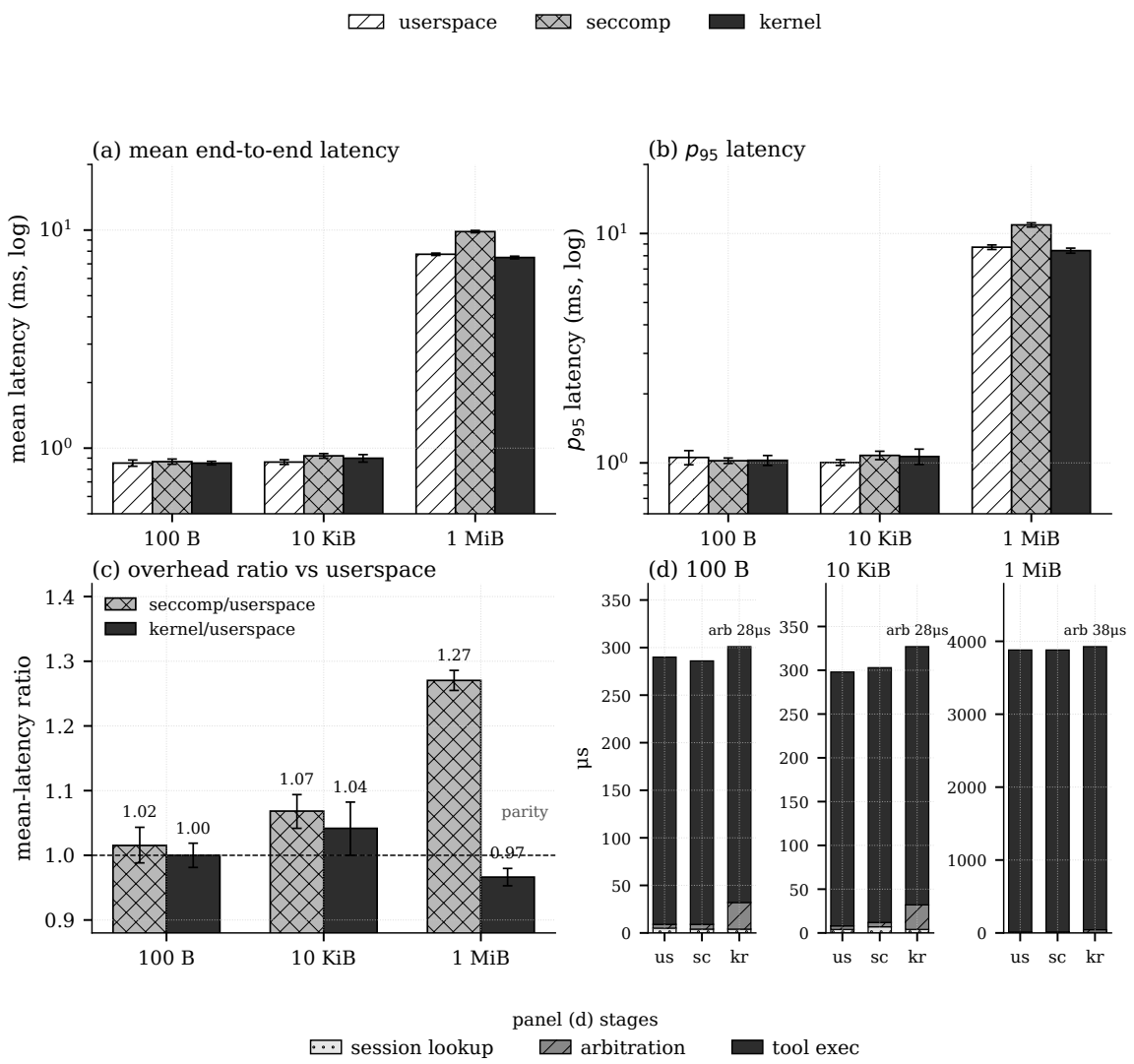


Figure 4: Latency summary across the three systems at 100 B, 10 KiB, and 1 MiB payloads ($n = 30$ independent runs of 2000 requests, error bars 95% CI). Panel (a): mean end-to-end latency on a log y-axis so sub-millisecond gaps at small payloads remain visible alongside the 1 MiB values. Panel (b): p_{95} latency; the three systems are indistinguishable at 100 B and 10 KiB, and the kernel p_{95} sits below both baselines at 1 MiB. Panel (c): mean-latency ratio against the `user-space` baseline, zoomed around the parity line; `seccomp` sits above parity at every payload size, while the kernel path only crosses below parity at 1 MiB. Panel (d): per-payload stage stack in μs, with three sub-axes auto-scaled per payload; each bar decomposes one system into session-lookup, arbitration, and tool-exec. The annotation above each kernel bar reports the absolute kernel arbitration cost (28 μs at 100 B and 10 KiB, 38 μs at 1 MiB), which is the only stage with a meaningful inter-system delta and remains roughly one order of magnitude below tool execution at small payloads (tool exec / arbitration $\approx 10\times$) and at least two orders of magnitude below at 1 MiB ($\approx 102\times$ against tool execution alone, and $\approx 197\times$ against full end-to-end latency).

millisecond within that bucket. Measured against the full end-to-end mean latency (7.487 ms on kernel, 7.748 ms on user-space), tool execution itself is closer to half (51.9% and 49.9% respectively) and the difference is unaccounted-for cost outside the instrumented stages (chiefly

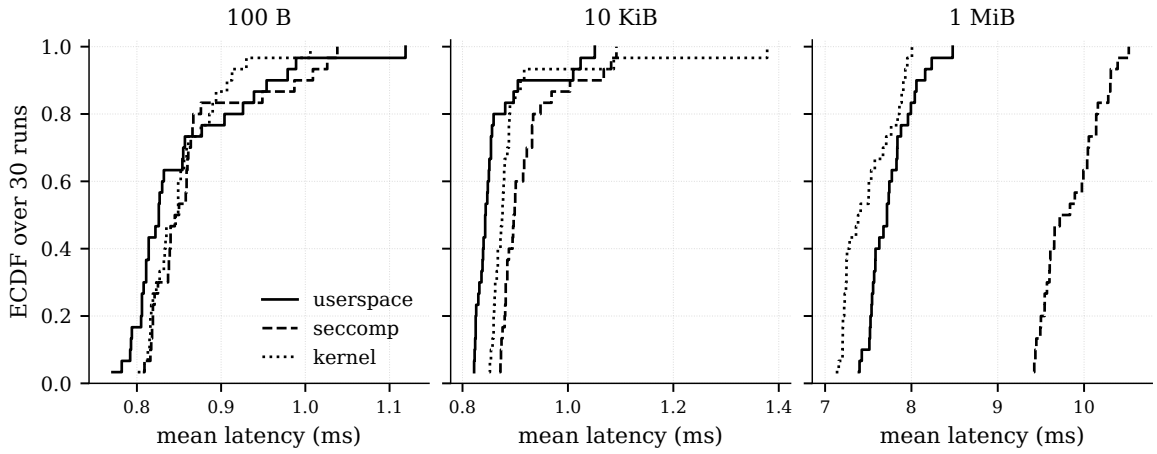


Figure 5: Empirical CDF of per-run mean latency ($n = 30$ runs per payload per system), with each subplot auto-scaled in x so the relevant regime is visible. Overlapping curves at 100 B and 10 KiB indicate that the per-run means are indistinguishable across systems; at 1 MiB the `seccomp` curve shifts rightward and the `kernel` curve lies to the left of `user-space`, with no heavy right tail on the kernel path.

the `mcpd` → tool-app UDS round-trip, JSON framing, and process scheduling), which scales with payload size and dominates the absolute number at 1 MiB.

Figure 5 complements the summary statistics with the empirical distribution of per-run mean latency at each payload size. The ECDF view shows the same three regimes as distributions instead of point estimates: at 100 B and 10 KiB the three curves overlap almost completely, indicating that the per-run means fall into the same distribution; at 1 MiB the `seccomp` curve shifts rightward by roughly 2 ms while the `kernel` curve lies to the left of `user-space`. No heavy right tail appears on the kernel path at any payload size, consistent with the p_{95} result.

A supplementary microbenchmark isolates the Generic Netlink layer using a minimum-cost `KMCP_CMD_NOOP` handler added to the kernel module. The NOOP path returns immediately after Generic Netlink demultiplexing, without touching any tool or agent registry, and therefore measures the irreducible floor of a round-trip against which all per-stage arbitration costs can be anchored. Figure 6 reports the medians on a single axis: the NOOP floor sits at 3.125 μs, the full `TOOL_REQUEST` path measured at a realistic registry scale of 1024 tools and 64 agents sits at 7.459 μs, and the `skip_lookups` and `ticket_trigger` bars bracket the per-stage contributions that the next paragraph decomposes. The total kernel arbitration cost is therefore 4.334 μs above the noise floor, against the 28–38 μs figure that the end-to-end stage breakdown attributes to "kernel arbitration" in panel (d). The order-of-magnitude gap reflects what each measurement covers: the microbenchmark times only the kernel-side handler, while the end-to-end breakdown also captures the user-space → kernel → user-space Generic Netlink round-trip, the Python framing inside `mcpd`, and the socket I/O for both the request and the reply. The dominant cost on the kernel path therefore comes from user-space components, not from the kernel module itself.

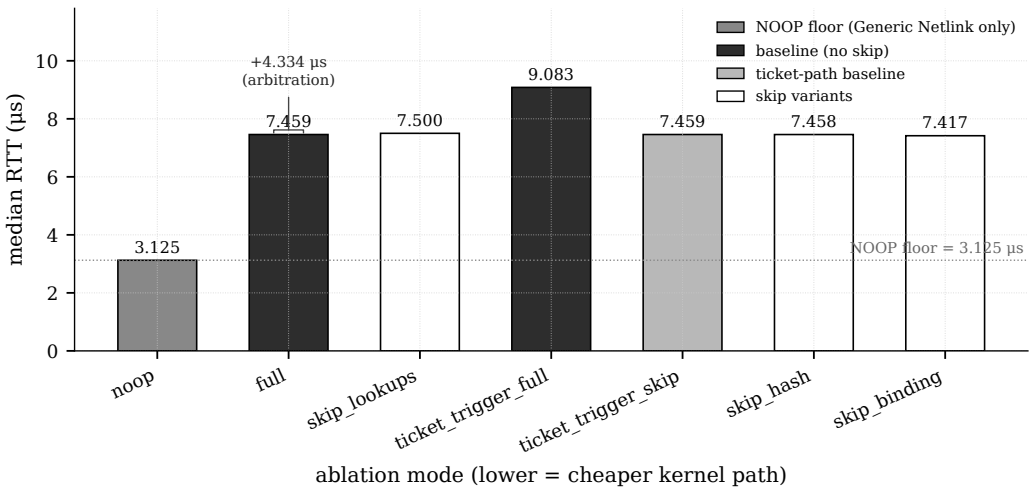


Figure 6: Kernel path ablation at realistic registry scale ($N = 1024$ tools, $M = 64$ agents). Bar heights report median RTT (p_{50}) over $n = 10$ repetitions (≥ 1000 requests per cell, ≥ 2000 for the NOOP floor); error bars are omitted because the CI95 column in the source data describes the mean rather than the median. The leftmost bar is the measured noise floor from the `KMCP_CMD_NOOP` handler, which establishes the irreducible Generic Netlink round-trip cost. The `full` bar is the complete `TOOL_REQUEST` arbitration path; `skip_lookups` disables the tool and agent registry walks; the two `ticket_trigger` bars exercise a high-risk tool whose execution is gated by the approval-ticket machinery, with and without the approval branch bypassed; `skip_hash` and `skip_binding` bypass the hash and binding-mismatch guard bodies (dead code under benign workload). The dotted line and inline annotation visualise the +4.334 μs arbitration delta over the NOOP floor that paragraph above quantifies.

To decompose the 4.334 μs arbitration cost into its constituent stages, a paired ablation was performed against compile-time experiment flags that selectively bypass individual kernel code paths (`SKIP_LOOKUPS`, `SKIP_HASH`, `SKIP_BINDING`, `SKIP_TICKET`). Each stage delta is measured by alternating the baseline and bypass modes request-by-request within a single benchmark phase, so the two samples in every pair are collected microseconds apart and share the same scheduler state and cache residency. Hypervisor-level drift that contaminates naive sequential-mean comparisons cancels out inside the paired difference, and bootstrap CIs on the delta vector are correspondingly tighter.

Table 7 and Figure 7 together report the two stages whose guard bodies are actually executed by the benchmark workload, plus two methodology-check rows that document the path-identity caveat. The `registry+agent_lookup` stage (two mutex pairs, an `xa_load` against a 1024-entry xarray, and a 64-entry hashtable walk) contributes 0.109 μs per request. The `approval_ticket_body` stage (consume plus issue, including an `approval_lock` acquisition and hashtable insert) contributes 2.012 μs per request and is only exercised when the tool is high-risk and the request arrives without a pre-authorised ticket. Under benign credentials the `hash` and `binding` guard bodies are dead code (short-circuited by the workload state before the experiment flag is consulted), and their paired deltas correctly return 0$\pm$ noise with $p_{\mathrm{BH}} > 0.5$, confirming that the paired method introduces no systematic bias on identical code paths.

Table 7: Paired per-stage ablation deltas. Each row alternates a baseline mode and a bypass mode request-by-request inside a single phase; the Δ column is the mean of $\text{baseline}_i - \text{bypass}_i$ across all pairs. BH-corrected paired t-tests are reported across all four comparisons.

Stage	n_{pairs}	Δ_{avg} (μs)	**95% CI** (μs)	t_{paired}	p_{BH}
registry+agent_lookup	100 000	$+0.109$	$[+0.087, +0.131]$	$+10.08$	$< 10^{-4}$
approval_ticket_body	1 000	$+2.012$	$[+1.760, +2.257]$	$+15.99$	$< 10^{-4}$
Methodology consistency checks (dead guard bodies under benign workload)					
hash_guard_body	100 000	$+0.163$	$[-0.153, +0.541]$	$+0.88$	0.507
binding_guard_body	100 000	~ 0.000	$[-0.027, +0.013]$	~ 0.00	0.513

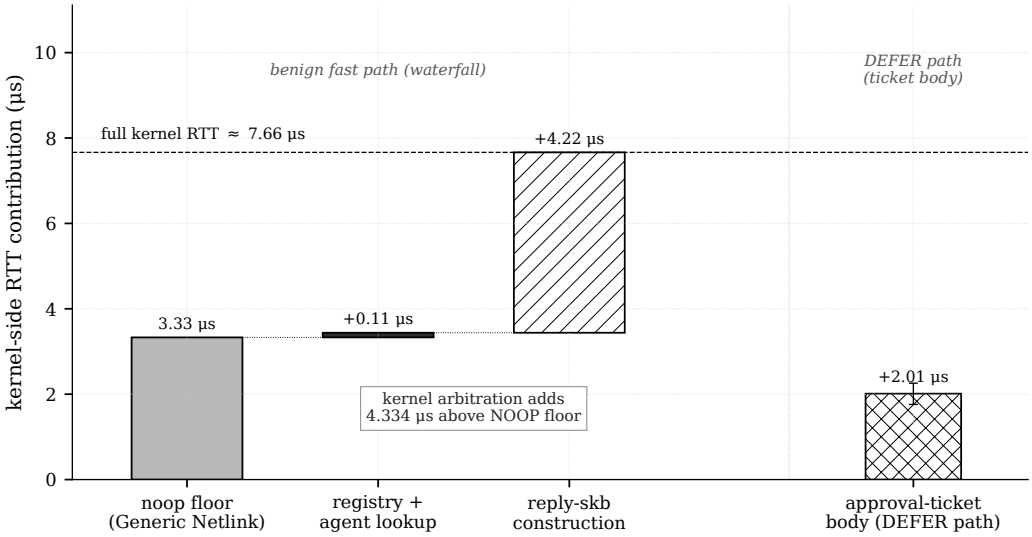


Figure 7: Paired per-stage deltas across the four ablation comparisons. The first two bars are the meaningful stage costs (registry+agent lookup and the approval-ticket body); the last two are methodology consistency checks against dead guard bodies and are expected to fall on zero.

Subtracting the 0.109 μs registry+lookup cost from the 4.334 μs total leaves roughly 4.2 μs unattributed in the benign run (the approval-ticket body is not exercised along this path, so its cost does not enter the sum). The remainder is reply-skb construction: the five `nla_put_*` calls and the `genlmsg_reply` in `kernel_mcp_reply_tool_decision`, which the experiment flags do not decompose further.

One caveat about the `approval_ticket_body` figure deserves explicit mention. Every ticket issuance calls `kernel_mcp_purge_expired_tickets_locked()`, which iterates the full 256-bucket approval hashtable. Under 300 s TTL, no ticket expires during a one-minute microbenchmark, so the purge scan cost grows linearly with the number of tickets the benchmark has already issued. Moving the expiry scan to the existing periodic `kernel_mcp_ticket_cleanup_timer` would eliminate this $O(n)$ component entirely. This is flagged as a simple future-work optimisation, not a characteristic of the fast path in production use where `mcpd` consumes tickets promptly and the hashtable stays small.

4.2 Throughput and Scalability

Throughput was measured across five agent counts (1, 5, 10, 20, and 50) and four concurrency levels (1, 10, 50, and 100) to check whether the kernel-assisted control plane becomes a bottleneck once multiple requests are in flight simultaneously. Table 8 gives the per-cell peak throughput at one agent; the complete sweep across agent counts produces the same shape and is summarised in prose below.

Table 8: Peak throughput at one agent, across concurrency levels ($n = 5$ repetitions per cell). Ratio columns report throughput relative to `user-space` on the same row. The kernel path tracks the user-space baseline within ∼10% at every concurrency level and does not collapse inside the tested window.

	user-space	**seccomp**		**kernel**	
Conc.	**RPS**	**RPS**	**rel.**	**RPS**	**rel.**
1	1117.5	1088.5	0.974×	1071.1	0.958×
10	1146.0	1036.9	0.905×	1009.4	0.881×
50	1215.6	1123.5	0.924×	1096.2	0.902×
100	1213.4	1072.5	0.884×	1103.7	0.910×

Plain user-space achieves the highest peak throughput across the full agent×concurrency sweep (1090–1219 RPS, plotted in Figure 8). Over the same 5×4 grid of agent counts and concurrency levels, the kernel path ranges from 1003 to 1137 RPS and the seccomp path from 1020 to 1200 RPS; Table 8 reports only the agents=1 row, so its kernel column (1009–1104 RPS) is a tighter slice of this wider range. At a representative operating point of one agent and concurrency 50, the three systems sit at 1215.6, 1123.5, and 1096.2 RPS respectively. The kernel path is roughly 10% slower than the user-space peak, which is consistent with the additional Generic Netlink round-trip and with the serialisation of admission decisions through a single kernel module. Figure 8 plots the complete agent×concurrency grid and shows that the relative ordering is stable across all five agent counts: the `user-space` line sits at the top, the `seccomp` and `kernel` lines track each other within a few percent, and none of the three curves bends downward as concurrency climbs from 1 to 100.

These numbers come from an earlier $n = 5$ pilot run. The $n = 30$ study that drives the latency table was scoped to latency alone and did not repeat the concurrency sweep, so the throughput figures carry wider uncertainty than the latency figures and should be read as evidence of practical viability, not as a precise ceiling. Within the tested range the kernel path stays within roughly 10% of the user-space peak without collapsing, which is enough to support the viability claim. A follow-up sustained-overload run was attempted at concurrencies of 200, 400, and 800, but the `SOMAXCONN` accept queue saturated before the control plane itself became the bottleneck, so those cells produced error-dominated latency distributions instead of usable p99 curves and are not reported here.

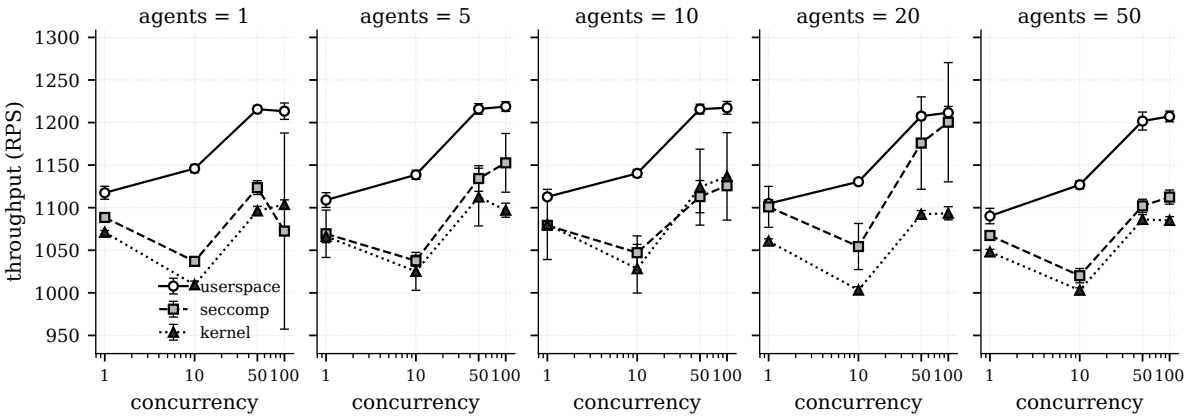


Figure 8: Throughput (RPS) across the full agent×concurrency sweep ($n = 5$ repetitions per cell, error bars 95% CI). Each panel fixes an agent count and varies concurrency on a log axis. The relative ordering is stable across all five panels, and none of the three systems collapses inside the tested window.

4.3 Registry Scaling

A separate experiment characterises how the kernel-side tool registry behaves as the catalogue grows beyond the 43-tool deployment used elsewhere in the evaluation. N synthetic tools (with $N \in \{8, 16, 32, \ldots, 16384\}$) are bulk-registered into the kernel, and both the register path and the steady-state lookup path are measured. The register path is modelled as a one-time setup cost plus a per-tool steady-state cost:

$$\text{total_ms}(N) = a + b \cdot N,$$

fitted across the 12 values of N by ordinary least squares.

The fit produces $a = -0.123$ ms and $b = 6.010$ μs per tool, with $R^2 = 0.9996$ on the linear total-time fit. That R^2 is close to 1 mostly because total_ms varies over three orders of magnitude with N and so reflects the dominance of the linear term, not the goodness of the per-tool view; a more direct test of the linear-scaling hypothesis is the log-log regression $\log(\text{total_ms}) = \alpha + \beta \log N$, which gives $\beta = 0.986$ with a bootstrap 95% CI of $[0.958, 1.014]$ that contains 1.0. This supports the claim that total registration cost scales linearly with N across the measured range, independent of the per-tool view. The asymptotic registration throughput is therefore $1/b \approx 166000$ tools/s; the 43-tool catalogue used in the main evaluation registers in approximately 0.14 ms of wall-clock time (extrapolated from the linear fit), and even a 16384-tool deployment completes its one-shot registration in under 100 ms. The per-tool cost (Figure 9) sits within roughly ± 1 μs of the asymptote across three orders of magnitude of N, and is indistinguishable from a constant for $N \geq 2^7$. The apparent excursion at small N is the intercept a amortising away rather than a scaling degradation.

The steady-state lookup path is measured separately by drawing a random `tool_id` for each

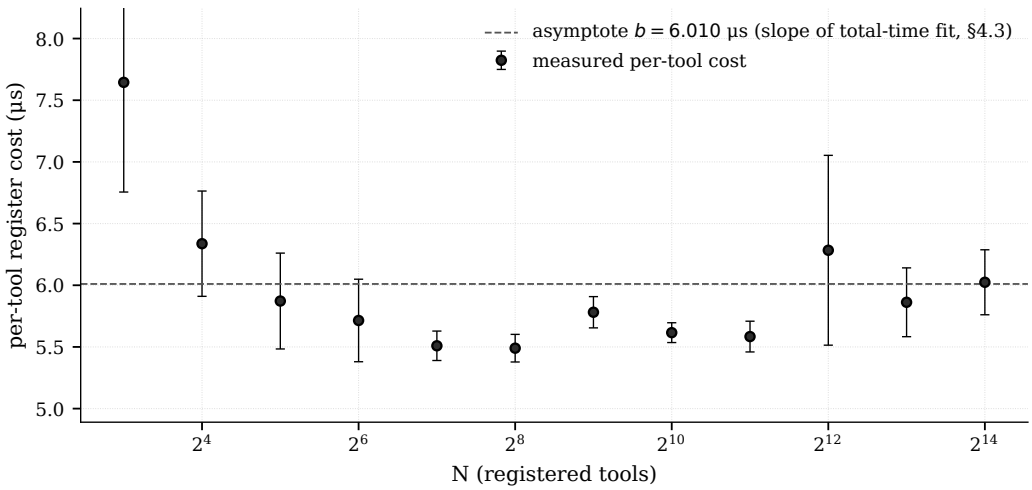


Figure 9: Registry register-path per-tool cost as a function of registry size N, with 95% CI error bars. The dashed line is the asymptotic per-tool cost $b = 6.010$ μs, taken as the slope of the linear fit total_ms$(N) = a + bN$ used in Section 4.3. We do not overlay the per-tool fit curve $a/N + b$: the OLS-derived intercept a is slightly negative (an artefact of the fit being dominated by large-N residuals in the millisecond range), so the curve would reach the asymptote from below while the data reaches it from above. Measured per-tool cost stays within roughly ± 1 μs of the asymptote across three orders of magnitude of N and is indistinguishable from a constant for $N \geq 2^7$.

request against the pre-populated registry; the log-linear regression lookup_ms $\sim a + b \cdot \log_2 N$ produces a slope of 2.9×10^{-5} ms per doubling with a bootstrap 95% CI of $[-2 \times 10^{-6}, +6.1 \times 10^{-5}]$ that contains zero ($R^2 = 0.26$). The near-zero slope and the CI overlapping zero are consistent with the expected $O(1)$ behaviour of the underlying xarray data structure, and inconsistent with any balanced-tree implementation where slope would grow with $\log_2 N$ at high R^2. Figure 10 shows the lookup latency distribution across the full range of N. Mean lookup latency stays within the 7.1–7.7 μs band across three orders of magnitude of registry size.

4.4 Security Evaluation

The latency and throughput numbers establish that kernel placement is not prohibitively expensive, but the argument for making the move hinges on whether it actually changes what attackers can do. The security evaluation targets four categories from the threat model in Chapter 2: identity spoofing (forging session identity to impersonate a registered agent), replay (reusing a stale approval ticket), substitution (tampering with `tool_id` or `tool_hash` to redirect execution), and escalation (bypassing the approval gate on a high-risk invocation).

Each system received 130 attack attempts in total (13 distinct cases $\times$ 10 repeats: 3 spoof, 5 replay, 3 substitution, 2 escalation). Outcomes are summarised in Table 9 and Figure 11. Before reading the headline numbers, one methodology note bears emphasis: the user-space rates reported here reflect the per-category instrumented weakening described later in this section;

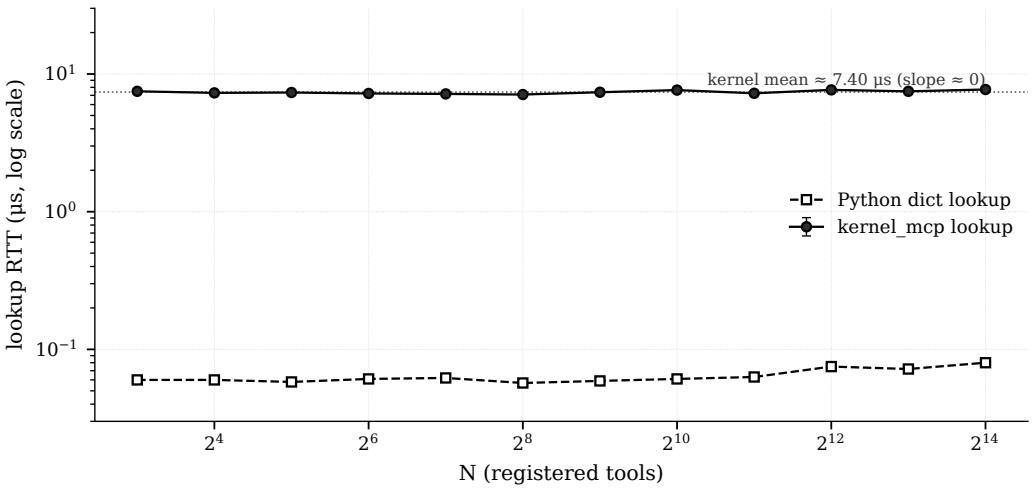


Figure 10: Steady-state `tool_id` lookup latency as a function of registry size N, on a log-N axis. The fitted log-linear slope is 2.9×10^{-5} ms per doubling (bootstrap 95% CI $[-2 \times 10^{-6}, +6.1 \times 10^{-5}]$, containing zero), consistent with the $O(1)$ xarray lookup invariant.

they are *not* the performance of a fully hardened user-space implementation.

Table 9: Category-level attack outcomes by system (130 attempts each: 3 spoof + 5 replay + 3 substitution + 2 escalation, $\times$ 10 repeats). "Bypasses" counts attempts that circumvented the intended control. 95% Clopper–Pearson confidence intervals on the bypass rate are reported in the last column. The 84.6% user-space rate is *not* a uniform per-case bypass: 11 of the 13 case rows are admitted in full (10/10 each = 110), while two cases (`spoof / session_token_theft` and `replay / cross_tool_ticket_reuse`) are blocked by orthogonal in-process checks even on the unhardened user-space mediator and contribute 0 admissions to the headline. The full per-case decomposition is plotted in Figure 11.

System	Bypasses	Attempts	Bypass rate	95% CI
`user-space`	110	130	84.6%	[77.2, 90.3]
`seccomp`	50	130	38.5%	[30.1, 47.4]
`kernel`	0	130	**0.0%**	[0.0, 2.8]

The plain user-space baseline is the weakest of the three: the harness selectively weakens its code paths per category (`tamper_session`, `tamper_approval`, `tamper_metadata`, `compromised_userspace`) so that each test exercises the specific check it targets, and the resulting bypass rate in Table 9 shows what a user-space mediator cannot structurally protect when its own state is under attack, not what a fully hardened version could achieve. The seccomp variant tightens checks and narrows the syscall surface but keeps admission as a user-space decision, so attacks that do not need to cross a privilege boundary (particularly substitution and spoofing) still get through.

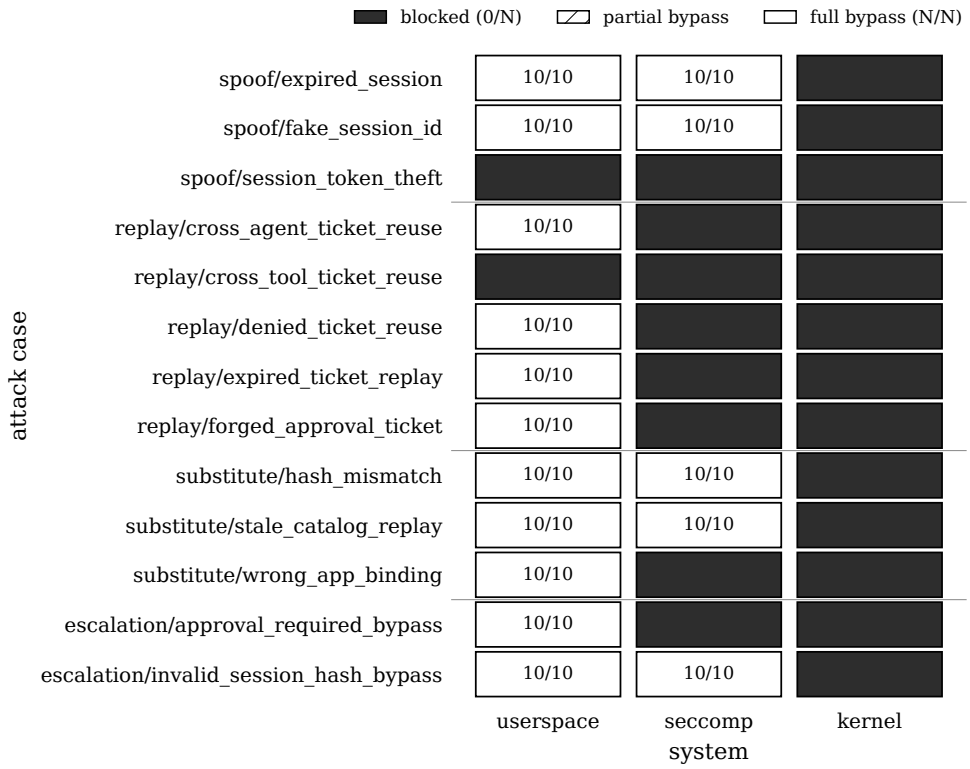


Figure 11: Attack bypass outcome per case and system (13 per-case rows grouped by attack type, 10 repetitions per cell). Dark cells indicate the attempt was fully blocked; white cells with a successes/attempts label indicate a full bypass; hatched white cells indicate a partial bypass. The `kernel` column is uniformly blocked across all 13 per-case rows, while the two user-space baselines permit bypasses on the specific checks that their instrumented tamper profiles relax.

The kernel path blocks every attempt in the table.

Spoofing fails because agent identity is tied to a kernel-registered identifier that the control plane verifies independently of the daemon's session state. Replay fails because approval tickets are kernel-resident lifecycle objects with explicit `consumed` and `expired` fields; once a ticket has been advanced or has passed its validity window, no subsequent `APPROVAL_DECIDE` can reopen it. Substitution is blocked by the semantic hash check: regardless of whether an attacker modifies the `tool_hash` field in the request or replaces the tool endpoint at runtime, the kernel compares the submitted hash against the one recorded at registration and rejects any mismatch. A supplementary injection experiment confirms this independently: in 30 live `llm-app` → `mcpd` → `kernel` execution chains, every runtime hash substitution produced a `DENY` with `reason=hash_mismatch` at the kernel layer, regardless of which tool the LLM planner had selected.

One implementation detail of the ticket lifecycle strengthens the replay defence beyond what the category table directly measures: a ticket is bound to the session, agent, tool, and hash, and also to the exact `req_id` of the issuing request. A replay consume attempt that reuses a different `req_id` returns `approval_ticket_scope_mismatch`, making it substantially harder to exploit a stolen ticket identifier in a fresh request context.

Two scoping limits bound the interpretation of Table 9. Each attack category maps one-to-one onto an enforcement mechanism in the kernel module, so the 0% bypass rate is a functional-correctness test of four specified invariants against inputs constructed to exercise exactly those invariants, not evidence of adversarial robustness against unknown attacks. The combined attack set is also scoped to the stated control-plane invariants and excludes attacks assuming compromise of the kernel module itself, attacks exploiting semantic vulnerabilities in tool logic, attacks requiring already-obtained privileged capabilities (such as `CAP_NET_ADMIN`), and attacks against the LLM prompt-construction pipeline. The two dominant threat classes in AI-agent deployments, prompt injection [11, 25] and supply-chain compromise of manifests, live at layers where kernel admission control has no visibility (the planner's reasoning layer and the filesystem integrity layer respectively) and are addressed by complementary defences rather than by this work.

4.5 Extended Threat Model

The 130-attempt category-level evaluation in Table 9 covers design-time invariants that the kernel module was built to enforce. A separate experiment targets three threat classes that sit outside that set and that a design-time review would not surface: a time-of-check-time-of-use (TOCTOU) race on the approval window, a cross-UID session hijack under adversarial Generic Netlink credentials, and dumb fuzzing of the `KERNEL_MCP` Generic Netlink family.

TOCTOU on the approval window. Two concurrent threads race against a high-risk tool: one repeatedly consumes approval tickets while the other re-registers the underlying tool binding, creating a window in which a ticket issued against one binding could in principle be consumed against another. Across 10 000 iterations, zero `ALLOW` outcomes were produced: 9985 requests were rejected with `hash_mismatch` at the kernel boundary, and the remaining 15 were deferred with `approval_ticket_unknown` because the consume attempt raced ahead of ticket issuance. The Clopper–Pearson 95% CI on the blocked rate is $[0.9996, 1.0000]$. The absence of breaches is consistent with the lock ordering in `kernel_mcp_cmd_tool_request`: the ticket consume and the tool-hash recomputation both acquire `tool_lock`, so a replacement that swaps the underlying binding cannot land between the two checks.

Cross-UID session hijack. A session is opened by UID A, and a second process running as UID B attempts three replay variants: blind replay, replay with a guessed session identifier, and replay with a leaked session identifier. Across 500 attempts, this experiment surfaced a defect that a design-time review of the kernel module had missed. In the version merged from the main evaluation branch, `kernel_mcp_cmd_tool_request` never compares `NETLINK_CB(skb)` peer credentials against the `uid` recorded at agent registration, so a crafted-credential replay from UID

B reaches the request handler as if it had been sent by UID *A*. Of the 500 attempts, 326 were still blocked by downstream `binding_mismatch` at the kernel boundary against the replayed request body, but 174 passed every kernel-side check that the stock binary enforces and would have been admitted to tool execution. Those 174 are what the zero-bypass numbers in Table 9 do not cover, because the category-level harness never probes a second UID.

The fix is a six-line patch that walks `sock_i_uid(NETLINK_CB(skb).sk)` on entry to the handler and rejects the request with `-EPERM` on mismatch, gated behind a new `require_peer_cred` module parameter so that the A/B comparison can be run against a single binary. With the gate enabled, all 500 attempts are blocked at peer-credential verification before any registry work runs; the Clopper–Pearson 95% CIs for the two conditions do not overlap. The patch adds no measurable latency because the peer-credential read is on the same cache line as the existing `NETLINK_CB` access that the handler already performs. Figure 12 visualises the A/B outcome: in the stock binary all 174 `leaked` attempts are admitted, while the same input set is uniformly blocked at `peer_cred_mismatch` once the gate is enabled. This evaluation-discovered defect changes the interpretation of the zero-bypass result in Table 9: the category-level harness did not cover cross-UID replay, and the stock binary did not enforce the peer-credential invariant until the E4 experiment exposed the gap.

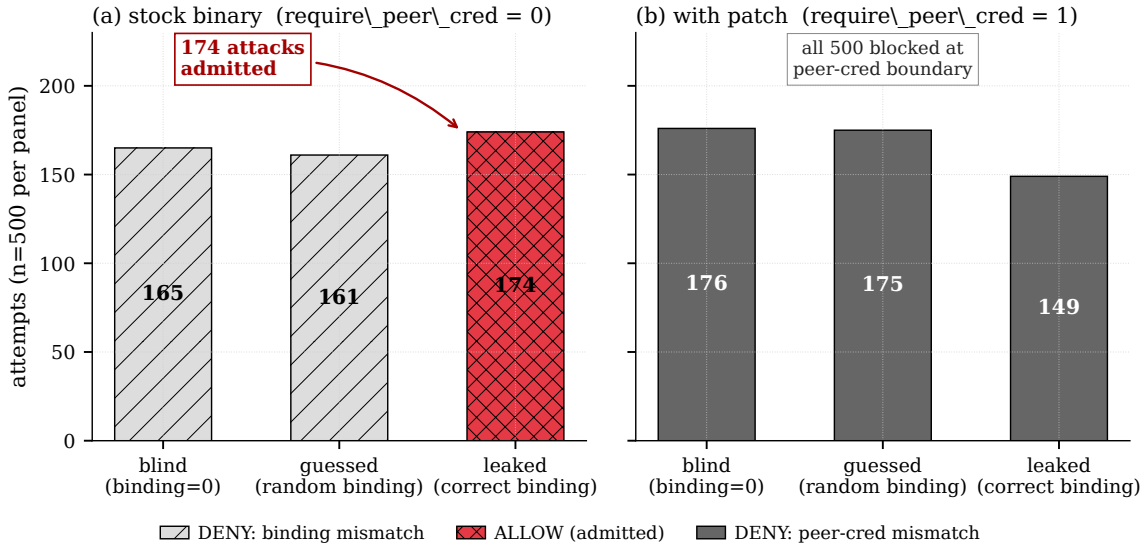


Figure 12: Cross-UID hijack outcomes by attack sub-case (`blind`, `guessed`, `leaked`) and binary state, $n = 500$ per panel. Panel (a) is the stock module: only the `leaked` sub-case (where UID *B* has obtained UID *A*'s 64-bit binding-hash cookie) reaches the request handler unchallenged, admitting all 174 attempts to tool execution. Panel (b) is the same module rebuilt with `require_peer_cred = 1`: all 500 attempts are rejected at the new peer-credential check before the request reaches the registry walks. The DENY-path median latency is unchanged between the two conditions, so the additional check imposes no measurable runtime cost.

Generic Netlink dumb fuzzer. The `KERNEL_MCP` family was exercised with a coverage-free structured fuzzer that iterates every defined `KMCP_CMD_*` opcode and applies per-`nlattr` bit flipping, length mutations, and type substitutions. Over a 30-minute fuzzing window, 880 640 mutated messages were delivered to the kernel module. Monitoring the kernel log buffer across the run (`dmesg` delta plus `kmemleak` scan) produced zero `oops`, `WARN`, `BUG`, `GPF`, or memory-leak reports. Every malformed input was rejected with an `errno` drawn from a bounded set of five expected classes (`-EPERM`, `-ENOENT`, `-EFAULT`, `-EINVAL`, `-ERANGE`); we do not claim per-input determinism (the harness did not replay individual inputs to verify same-input/same-errno behaviour), only that no input fell outside this set. The fuzzer's design time was limited, so the result is a robustness smoke test, not evidence of adversarial hardness: it says the Generic Netlink dispatch layer does not crash on malformed input within the tested 30-minute budget, not that the kernel module is free of latent memory-safety bugs under an unbounded adversary.

4.6 Daemon Failure and Recovery

A core architectural argument for kernel-held state is that it should outlive the process that registered it. To test this property, the daemon was forcibly interrupted mid-operation and control-plane state was observed before and after restart.

In both user-space systems, daemon failure is control-plane failure. Registered tool state, in-flight sessions, and pending approval tickets are all bound to the process that held them. When it dies, they are gone. Recovery means starting over or reloading from application-level logs, with no consistency guarantee.

The kernel path is different. Approval tickets, the tool registry, and agent registrations survive the crash intact and remain inspectable via sysfs without any daemon running. Session state is not preserved; the binding context lives in daemon memory and is inherently tied to an active connection.

The durability claim here is observability-oriented, not warm-continuation-oriented: while `mcpd` is down, sysfs stays readable so an operator can still inspect the surviving registry, ticket state, and per-agent `call_log` entries. On daemon restart, the cold-start path described in Section 3.9 issues `RESET_TOOLS` and re-registers each manifest tool unconditionally, so the surviving kernel state serves audit rather than warm continuation; the conservative rebuild avoids relying on a residual view whose freshness the daemon cannot prove. Incremental reconciliation is already used while `mcpd` is running (only tools whose semantic or binding fingerprint changed are re-registered); extending that behaviour to the cold-start path, gated on a divergence signal, would reduce restart overhead and is a natural direction for future work.

4.7 Discussion

Seccomp does more work than plain user-space, including tighter syscall filters, extra validation, and audit logging, yet it still admits 38.5% of attacks (Table 9) and is the slowest system at megabyte payloads (Table 6). The kernel path blocks every attempt because the admission state sits where no unprivileged process can reach it; seccomp keeps filtering around the same mutable process that holds the decision.

The 1 MiB latency picture is the same architectural fact viewed from the performance side: kernel arbitration runs on a fixed-size Generic Netlink message regardless of payload, while seccomp's BPF filter pays per syscall and so accumulates cost across the heavier socket traffic a megabyte payload forces through. The kernel path's 3.4% latency reduction over plain user-space at 1 MiB (Table 6) is statistically clean but not clearly attributable from the available stage instrumentation; we read it as evidence of no measurable overhead at this payload size, not as evidence of a positive speedup mechanism. The ablation breakdown agrees: of the 4.334 μs that kernel arbitration adds over the NOOP floor, registry+lookup costs 0.109 μs and the rest is reply-skb construction. Nothing in the fast path scales with tool count or request body, and the registry scaling experiment extends this end-to-end: lookup latency stays within a 0.6 μs band from $N = 8$ to $N = 16\,384$, and registration runs at roughly 166 000 tools per second.

Cross-UID (Section 4.5) is the clearest reminder that the zero-bypass table measures only design-time invariants: it exposed a peer-credential gap the category-level harness never probed, closed by a six-line patch at no measurable latency cost; the design-time and extended-attack layers are complementary, and neither alone would have caught this.

Within the swept window of up to 100 concurrent requests the kernel path holds within roughly 10% of plain user-space throughput; beyond it, the global registry lock is the obvious suspect and per-tool or per-agent lock decomposition the natural fix, but both lie outside the scope of this evaluation.

4.8 Threats to Validity

The results should be interpreted within four scope limitations. First, all experiments ran in a single VMware guest on an aarch64 host, so latency figures are environment-specific and generalisation to bare-metal or different kernel configurations is not established. Second, the evaluation uses demonstration tools with predictable execution times and does not characterise tail latency under sustained load. Third, the attack study is a controlled functional-correctness test of the four explicitly identified invariants and does not constitute a robustness assessment against unknown attacks, prompt injection, or privileged adversaries. Fourth, kernel-level admission control addresses only one layer of an AI-agent system, leaving semantic tool vulnerabilities, unsafe policy choices, and compromise outside the stated threat model unaddressed.

Chapter 5: Conclusion and Further Work

5.1 Conclusion

The broader question behind this project is what a traditional Linux system needs in order to host AI agents well, asked narrowly through MCP: which parts of a tool-invocation control plane need operating-system support, and which are fine in user space? For the control-plane side the answer is a small set of invariants (semantic consistency between a registered tool and the one actually executed, identity binding between a request and its session, approval validity across the defer-approve-execute lifecycle, and state durability across gateway failure), each structurally hard to hold in a user-space gateway, because each has to survive the failure or compromise of the very process that enforces it.

linux-mcp is one concrete shape this split can take: a small kernel module holds registries, admission decisions, and the approval-ticket lifecycle behind a `CAP_NET_ADMIN`-gated Netlink interface, while `mcpd` handles manifests, semantic hashes, session binding, and forwarding. `mcpd` is trusted to populate the kernel's view at initialisation and to forward Netlink frames faithfully thereafter, but its runtime logic (session state, ticket routing, request forwarding) is no longer load-bearing for individual request validation; a memory-safety compromise of the daemon itself remains out of scope, since `mcpd` still holds `CAP_NET_ADMIN`, and we discuss a minimal split of that capability in Section 5.3. In the tested prototype, the kernel path enforced all specified control-plane invariants across the targeted attack set while both user-space baselines permitted meaningful bypass rates, with latency and throughput close to the user-space baseline throughout; a subsequent extended-attack rerun surfaced a missing peer-credential check in the stock module, closed by a six-line patch at no measurable latency cost.

The position is narrower than AIOS-style proposals that rebuild OS infrastructure around LLM abstractions, and less trusting than user-space frameworks that accept gateway correctness as given. Whether AI agents will eventually warrant an operating system of their own remains an open question; in the meantime, a conventional Linux system extended with a few carefully scoped agent-aware mechanisms can already carry an MCP control plane that neither a user-space gateway nor existing Linux security primitives can match alone.

These control-plane benefits are not free at deployment time, and an honest account should name the costs that do not show up in the latency and throughput numbers. Shipping a loadable kernel module introduces operational overhead that a user-space daemon avoids: a target-kernel version to track (the prototype is built against Linux 6.8 and compatibility across the active LTS range 5.15–6.12 has not been characterised), module-signing and loader-policy requirements on locked-down distributions, and a debugging path (`dmesg`, `kdb`, crash dumps) that most application engineers are unfamiliar with. For deployments where user-space hardening is already sufficient, these overheads may outweigh the microsecond-scale gains reported here, and a re-

alistic adoption path would likely require distribution-packaged modules, a documented kernel-version support matrix, and the `CAP_NET_ADMIN` split discussed in Section 5.3 before the design is operationally comparable to pure user-space alternatives.

5.2 Reflection

The hardest part of this project was not writing the kernel code, but deciding what the kernel should not do. For about the first six weeks, I treated AI-native OS support as a broad goal. As a result, the design kept expanding: more in-kernel state, more policy hooks, and broader claims about what the kernel should provide for agents. The project only became manageable when I narrowed it to four properties that a user-space gateway cannot reliably preserve on its own: tool identity, the approval-ticket lifecycle, session binding, and state durability across daemon failure. Everything else had to stay out. The rewrite after that decision was much faster, which in hindsight was a sign that the earlier version was not solving a harder problem, but a less clearly defined one.

I also spent roughly two weeks on the wrong kernel interface. The first serious candidate was eBPF rather than Generic Netlink. At first it looked attractive: pinned maps and `bpf_timer` could have represented the required state. The problem was the attachment point. eBPF LSM hooks fire on kernel operations, not on parsed JSON-RPC messages, so an in-kernel arbitrator would have had to reconstruct MCP framing from raw socket traffic inside a verifier-bounded program on every invocation. Plain LSM hooks had a similar problem: they run at the syscall boundary, where the notion of a tool does not yet exist. Generic Netlink was the first interface that matched the granularity of the design, so I accepted the cost of a loadable kernel module. That choice introduced its own problems. Attribute-layout mismatches between the Python side and the kernel side produced malformed requests without compiler warnings or useful logs. After losing nearly two days to one such mismatch, I started writing small per-command Python harnesses to construct and test each Netlink frame explicitly.

Another recurring temptation was to move more validation into the kernel. Whenever I saw a check that a compromised `mcpd` might skip, the natural reaction was to duplicate it inside the module. I made that mistake once with partial schema validation at registration time. It worked until the schema changed; then a simple user-space update turned into a kernel rebuild and reload, while still duplicating logic that `mcpd` needed anyway. Removing that code led to one of the more important design rules of the project: the kernel should verify *membership*, but it should not compute *content*. This rule was not planned from the beginning. It came from undoing a concrete mistake. Without it, the module would have slowly grown into a second tool-execution engine inside the kernel, because every individual bug seemed to justify just one more kernel check.

The evaluation phase was the part that most changed how I think about systems experiments.

An early attack suite reported clean passes for the cross-UID hijack cases. The result looked encouraging for several days, until I found that the kernel was not checking peer credentials on the `TOOL_REQUEST` path at all. The tests were passing because the harness was not exercising the path its name claimed to test. The actual fix was small: compare the request peer uid with the uid bound to the session. It added no measurable latency, but the uncomfortable lesson was that the earlier result had looked plausible enough to escape scrutiny. A separate overload issue taught the same lesson in the opposite direction. I initially blamed a kernel-side limit, but the real cause was `SOMAXCONN=128` saturation in the harness. The habit I would carry into a future project is simple: a clean security-test pass is not enough. I should first instrument the path and confirm that the intended check was actually reached.

5.3 Further Work

The directions that follow are organised from short-range engineering items, which would consolidate the prototype into a deployment-ready system, to longer-range research questions for which the present design is closer to a starting point than a finished answer.

On the engineering side, several items remain inside the existing design envelope. The kernel registries are protected by a single global lock that becomes a scalability bottleneck at higher concurrency and would benefit from RCU-protected lookup on the request path; hot reload is request-driven (`mcpd` recomputes a manifest-set signature on every request) rather than filesystem-driven, leaving a bounded staleness window between manifest edit and uptake that an `inotify`-driven path would close; cold-start reconciliation is unconditional rather than gated on an observable divergence signal; and the trust boundary around `mcpd` can be tightened by splitting `CAP_NET_ADMIN` into a minimal forwarder service that only relays pre-validated frames, which is consistent with the narrow-kernel principle and reduces the blast radius of a daemon compromise without widening the kernel interface. The evaluation envelope has matching engineering items: bare-metal characterisation to eliminate hypervisor timing noise, p_{99} under sustained concurrency past the clean-sweep ceiling (which requires repairing the `SOMAXCONN` saturation in the harness first), heterogeneous tool workloads with network-dependent execution times, and comparison against additional hardened user-space designs to separate benefits of kernel placement from those obtainable by user-space engineering alone. None of these items changes the design narrative; they would close the gap between the prototype's measured behaviour and what a production deployment would need.

Beyond consolidation, four directions are closer to open research questions, where the present design is more naturally read as a starting point than a finished answer.

The first is a minimality argument for the kernel surface itself. The present work picks four invariants by inspection (tool identity, approval-ticket lifecycle, session binding, state durability across daemon failure) and shows experimentally that a user-space gateway fails on each. A

stronger claim would establish a lower bound: under a stated threat model that admits memory-safety compromise of the gateway, prove that no purely user-space design can preserve all four properties simultaneously, and characterise the proposed kernel surface as not strictly larger than necessary.

The second is in-kernel, agent-attributed accounting and cooperative scheduling for tool execution. The present prototype arbitrates whether a request runs but does not constrain how it runs once approved. A natural next step is to extend the per-agent state already held in the kernel into a cgroup-style accounting domain that tracks tool-execution budgets (wall time, fan-out, tool-class quotas) and enforces them at request admission rather than after the fact, with multi-agent fairness guarantees backed by the same kernel state that already binds sessions. This connects directly to the operating-system literature on resource-attribution containers, real-time scheduling, and BPF-CPU, and yields a falsifiable claim of the form "two agents sharing a host see tool-execution latency that diverges by no more than Δ under contention".

The third is attested arbitration. At present, the kernel trusts `mcpd`'s view of the manifest set at registration time, and any remote party trusts the kernel's word on what was registered. Anchoring the manifest-set hash in a TPM measurement and exposing a remote-attestation path that proves to an auditor which tool catalogue the kernel approved would close the gap between local enforcement and remote verifiability, which lines up with the broader confidential-computing direction in systems work. The existing design already produces the necessary hashes; what is missing is a verifiable chain from kernel state through measured boot to a remote claim, together with a defensible answer for how that chain survives module unload, kernel live-patch, and approval-ticket reissue.

The fourth is multi-host federation. The present design is single-machine, and any production deployment of LLM agents will span multiple hosts behind a fleet manager. A federated variant in which kernel-anchored state is replicated across hosts via an attested gossip protocol, with a defined consistency model under partition (eventually-consistent registries, strongly-consistent approval tickets), would extend the narrow-kernel principle into a distributed-systems setting and address what is, today, the most obvious deployment-scale objection to the design. Whether the kernel state needs to participate in the replication path or only sign for it, and whether approval-ticket linearisability requires consensus or can be reduced to leader-pinned issuance, are themselves the substantive research questions.

These four directions share a common shape: they take the current design's narrow kernel surface as a fixed boundary and ask what additional guarantees the user-space side can be given, rather than what additional logic the kernel can absorb. That orientation is what makes the design extensible without violating the narrow-kernel principle, and is, more than any individual direction listed here, the contribution this work hopes to leave behind.

References

[1] Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: ICLR; 2023. ArXiv:2210.03629.

[2] Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Hambro E, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. In: NeurIPS; 2023. ArXiv:2302.04761.

[3] Park JS, O'Brien J, Cai CJ, Morris MR, Liang P, Bernstein MS. Generative Agents: Interactive Simulacra of Human Behavior. In: UIST; 2023. p. 1-22.

[4] Qin Y, Liang S, Ye Y, Zhu K, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In: ICLR; 2024. ArXiv:2307.16789.

[5] Model Context Protocol. Architecture Specification; 2025. MCP specification revision 2025-06-18; accessed April 2026. `https://modelcontextprotocol.io/specification/2025-06-18/architecture`.

[6] Packer C, Fang V, Patil SG, Lin K, Wooders S, Gonzalez JE. MemGPT: Towards LLMs as Operating Systems; 2023. Preprint, arXiv:2310.08560.

[7] Mei K, Zhu X, Xu W, Hua W, Jin M, Li Z, et al.. AIOS: LLM Agent Operating System; 2024. Preprint, arXiv:2403.16971.

[8] Steinberger P. OpenClaw: Open-source Autonomous AI Agent; 2026. Project website; accessed April 2026. `https://openclaw.ai`.

[9] Nous Research. Hermes Agent; 2026. GitHub repository, accessed April 13, 2026. `https://github.com/nousresearch/hermes-agent`.

[10] Peter S, Li J, Zhang I, et al. Arrakis: The operating system is the control plane. In: OSDI; 2014. p. 1-16.

[11] Abdelnabi S, Greshake K, Mishra S, Endres C, Holz T, Fritz M. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In: AISec @ CCS; 2023. p. 79-90.

[12] Tanenbaum AS, Bos H. Modern Operating Systems. 5th ed. Pearson; 2022.

[13] Chase H. LangChain; 2022. Open-source LLM application framework; accessed April 2026. `https://github.com/langchain-ai/langchain`.

[14] LangChain AI. LangGraph: A Framework for Building Stateful Multi-Actor LLM Applications; 2024. Open-source agent orchestration framework; accessed April 2026. `https://github.com/langchain-ai/langgraph`.

[15] Gravitas S. AutoGPT: An Autonomous GPT-4 Experiment; 2023. Open-source autonomous agent framework; accessed April 2026. https://github.com/Significant-Gravitas/AutoGPT.

[16] CrewAI Inc . CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents; 2024. Open-source multi-agent collaboration platform; accessed April 2026. https://crewai.com.

[17] Microsoft Corporation. Semantic Kernel: Open-Source SDK for LLM Integration; 2023. Released publicly March 2023; reached v1.0 in early 2024; accessed April 2026. https://github.com/microsoft/semantic-kernel.

[18] Wu Z, Han C, Ding Z, Weng Z, Liu Z, Yao S, et al.. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement; 2024. Preprint, arXiv:2402.07456.

[19] Liu R, Zhe T, Wang D, et al.. AgentOS: From Application Silos to a Natural Language-Driven Data Ecosystem; 2026. Preprint, arXiv:2603.08938.

[20] HKUDS. OpenHarness: Open Agent Harness with a Built-in Personal Agent—Ohmo!; 2026. Initial open-source release v0.1.0 April 1, 2026; accessed April 2026. https://github.com/HKUDS/OpenHarness.

[21] Harness Inc . Harness AI DevOps Agent; 2025. AI-powered autonomous workers embedded in governed CI/CD pipelines; accessed April 2026. https://developer.harness.io/docs/platform/harness-aida/ai-devops/.

[22] Apple Inc . Private Cloud Compute: A new frontier for AI privacy in the cloud; 2024. Apple Security blog post; accessed April 2026. https://security.apple.com/blog/private-cloud-compute/.

[23] MCP Core Maintainers. One Year of MCP: November 2025 Spec Release; 2025. Model Context Protocol Blog; accessed April 2026. https://blog.modelcontextprotocol.io/posts/2025-11-25-first-mcp-anniversary/.

[24] Patil SG, Zhang T, Wang X, Gonzalez JE. Gorilla: Large Language Model Connected with Massive APIs. In: NeurIPS; 2024. ArXiv:2305.15334.

[25] Liu Y, et al.. Prompt Injection Attack against LLM-integrated Applications; 2023. Preprint, arXiv:2306.05499.

[26] Wang Z, Tu H, Zhang L, et al.. Your Agent, Their Asset: A Real-World Safety Analysis of OpenClaw; 2026. Preprint, arXiv:2604.04759.

[27] Wang Y, Gao H, et al.. A Systematic Security Evaluation of OpenClaw and Its Variants; 2026. Preprint, arXiv:2604.03131.

[28] Wang G, Xie Y, Jiang Y, Mandlekar A, Xiao C, Zhu Y, et al.. Voyager: An Open-Ended Embodied Agent with Large Language Models; 2023. Preprint, arXiv:2305.16291.

[29] Yang J, Jimenez CE, Wettig A, Lieret K, Yao S, Narasimhan K, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In: NeurIPS; 2024. ArXiv:2405.15793.

[30] Lim SY, Agrawal S, Han X, Eyers D, O'Keeffe D, Pasquier T. Securing Monolithic Kernels using Compartmentalization; 2024. Preprint, arXiv:2404.08716.

[31] Schneider M, Masti RJ, Shinde S, Capkun S, Perez R. SoK: Hardware-supported Trusted Execution Environments; 2022. Preprint, arXiv:2205.12742.

[32] Lefeuvre H, Dautenhahn N, Chisnall D, Olivier P. SoK: Software Compartmentalization. In: IEEE S&P; 2025. p. 3107-26.

[33] Ji Z, Wu D, Jiang W, Ma P, Li Z, Gao Y, et al.. Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework; 2026. Preprint, arXiv:2601.11893.

[34] Shi T, He J, Wang Z, Wu L, Li H, Guo W, et al.. Progent: Programmable Privilege Control for LLM Agents; 2025. Preprint, arXiv:2504.11703.

[35] Linux Kernel Documentation. Seccomp BPF;. Accessed April 2026. `https://docs.kernel.org/userspace-api/seccomp_filter.html`.

[36] Linux Kernel Documentation. Linux Security Modules (LSM);. Accessed April 2026. `https://docs.kernel.org/admin-guide/LSM/index.html`.

[37] Linux Kernel Documentation. AppArmor;. Accessed April 2026. `https://docs.kernel.org/admin-guide/LSM/apparmor.html`.

[38] Linux Kernel Documentation. SELinux;. Accessed April 2026. `https://docs.kernel.org/admin-guide/LSM/SELinux.html`.

[39] Linux Kernel Documentation. Landlock;. Accessed April 2026. `https://docs.kernel.org/userspace-api/landlock.html`.

[40] Wright C, Cowan C, Smalley S, Morris J, Kroah-Hartman G. Linux Security Modules: General Security Support for the Linux Kernel. In: Proceedings of the 11th USENIX Security Symposium. USENIX Association; 2002. p. 17-31.

[41] Salaün M. Landlock LSM: Toward Unprivileged Sandboxing; 2017. Linux Security Summit talk.

[42] Watson RNM, Anderson J, Laurie B, Kennaway K. Capsicum: Practical Capabilities for UNIX. In: Proceedings of the USENIX Security Symposium; 2010. p. 29-46.

[43] Brimhall B, Garrard J, de la Garza C, Coffman J. A Comparative Analysis of Linux Mandatory Access Control Policy Enforcement Mechanisms. In: Proceedings of the 16th European Workshop on System Security (EuroSec); 2023. p. 1-7.

[44] Bishop M. Computer Security: Art and Science. Addison-Wesley; 2003.

[45] Lee Y, Min C, Lee B. ExpRace: Exploiting Kernel Races through Raising Interrupts. In: Proceedings of the 30th USENIX Security Symposium. Vancouver, BC; 2021. p. 2363-80.

[46] Stevens WR, Rago SA. Advanced Programming in the UNIX Environment. 3rd ed. Addison-Wesley; 2013.

[47] Klein G, Andronick J, Elphinstone K, Heiser G, Cock D, Derrin P, et al. seL4: Formal Verification of an Operating-System Kernel. Communications of the ACM. 2010;53(6):107-15.

[48] Linux Kernel Documentation. Netlink Socket Family;. Accessed April 2026. `https://docs.kernel.org/userspace-api/netlink/intro.html`.

[49] West RM. Best practice in statistics: Use the Welch t-test when testing the difference between two groups. Annals of Clinical Biochemistry. 2021;58(4):267-9.

Acknowledgment

I would like to express my heartfelt gratitude to everyone I have met over the past four years at university. Their kindness, encouragement, and quiet influence have shaped this project, and the person I have become along the way.

Appendices

Disclaimer

This report is submitted as part requirement for the undergraduate degree programme at Queen Mary University of London, and Beijing University of Posts and Telecommunications. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

BUPT No.: 2022213192

QMUL No.: 221166471

Full Name (Pin Yin): Xiheng LI

Full Name (Chinese): 李溪恒

Signature:

Date:27-04-2026

Acknowledgement of using Generative AI

I acknowledge the use of the following Generative AI tools:

Publisher	**Name, version, URL**	**Use**	**Section**
OpenAI	Codex (GPT-5-based coding assistant) `https://openai.com/` ChatGPT (GPT-5.4) `https://chatgpt.com/`	Used for translation support during report preparation.	Chapter 1; Chapter 2; Chapter 3; Part of the source code
Anthropic	Claude Code (claude-sonnet-4-6) `https://claude.ai/code`	Used for grammar editing, and academic writing support during report preparation. Also, part of the scripts and source code are done by vibe-coding using Claude Code.	Chapter 1; Chapter 4; Part of the source code

I confirm the submitted work is my own work.

Project specification

北京邮电大学 本科毕业设计（论文）任务书

Project Specification Form

Part 1 - Supervisor

论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol		
题目分类 Scope	Research	Software Development	Software
主要内容 Project description	As AI agents increasingly function as autonomous computing entities capable of learning, reasoning, and collaboration, the fundamental limitations of traditional operating systems (OS) have become apparent, creating a need for a new generation of agent-oriented OS. This thesis focuses on developing a prototype of an agent-native OS centered around the Model Context Protocol (MCP). Specifically, it proposes an MCP-enabled kernel module that provides unified interfaces for AI agents to register, exchange contextual information, and coordinate computational tasks. Furthermore, it investigates how MCP-based kernel-level scheduling mechanisms can manage computational resources among multiple agents to ensure fairness, efficiency, and security. The results aim to lay the groundwork for future operating systems in which AI agents act as first-class citizens within the computing environment.		
关键词 Keywords	AI Agent, MCP, kernel		
主要任务 Main tasks	1 Analyze the evolution of OS architectures from process-centric to agent-centric paradigms and identify the key design principles of agent-native OS.		
	2 Study the internal structure of the MCP protocol and design how to provide its functionalities from the kernel level.		
	3 Implement a MCP-enabled kernel module and design the exposed APIs.		
	4 Evaluate the performance and scalability of the MCP-enabled kernel model, and discuss how it achieves efficiency, fairness, and security.		
主要成果 Measurable outcomes	1 A comprehensive review chapter comparing traditional OS abstractions with emerging requirements on OS from AI agents.		
	2 A detailed design discusses how MCP interfaces can be mapped to system calls or other interfaces the kernel exposes to user-level applications, and how MCP functionalities are implemented in the kernel.		
	3 Implement a working prototype of the MCP-enable kernel module, along with API documentation describing the exposed interfaces.		

北京邮电大学 本科毕业设计（论文）任务书

Project Specification Form

Part 2 - Student

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				
论文概述 Project outline **Write about 500-800 words** **Please refer to Project Student Handbook section 3.2**	This project aims to explore the design of an AI-Native Operating System (AI-Native OS) based on the Model Context Protocol (MCP). The system integrates a kernel-level MCP module, enabling native agent registration, context exchange, and scheduling. Through the integration of these layers, the project seeks to build a secure, efficient, and extensible foundation for next-generation operating systems where AI agents are treated as first-class computing entities. This project bridges MCP-based agent systems and next-generation operating system design, integrating secure automation with kernel-level scheduling. It aims to establish a secure and scalable foundation for future AI-native computing environments. The project addresses the needs of OS researchers and AI developers who require: 1. Unified interfaces for agent registration and context sharing; 2. Extensible OS infrastructure for managing heterogeneous agents. The developer feedback regarding system usability, safety, and performance. The project adopts a bottom-up system design combining kernel engineering and AI-driven automation. Key methodologies include: 1. Implementation of agents registration, context sharing, and cooperative scheduling. 2. Scheduling algorithm to balance fairness and resource efficiency using contextual metadata. The experiments will test the following hypotheses: 1. MCP-based kernel scheduling enhances fairness and efficiency compared with traditional schedulers. 2. The integrated kernel–server architecture provides lower latency and higher scalability than user-space MCP implementations. 3. Performance metrics include latency, CPU utilization, security violation rate, and scalability under multi-agent workloads. The experiments will be conducted in the following environment: 1. Programming Languages: TBD (kernel module), Python (testing and generation) 2. Software Tools: GCC, Clang, Docker, GDB, FastMCP, LangChain 3. Operating Environment: Linux Kernel 6.x (Ubuntu 22.04) 4. Hardware: x86-64 workstation or virtual machine				

	Background materials Zhang X, Zhang Y. Planet as a Brain: Towards Internet of AgentSites based on AIOS Server. arXiv preprint arXiv:2504.14411; 2025. Available from: https://arxiv.org/abs/2504.14411 Hu X, Xiong T, Yi B, et al. OS Agents: A Survey on MLLM-based Agents for General Computing Devices Use. arXiv preprint arXiv:2508.04482; 2025. Available from: https://arxiv.org/abs/2508.04482 Koubaa A. Agent Operating Systems (Agent-OS): A Blueprint Architecture for Real-Time, Secure and Scalable AI Agents. TechRxiv; 2025. Available from: https://www.techrxiv.org/users/689908/articles/1330306 Mastouri M, Ksontini E, Kessentini W. Making REST APIs Agent-Ready: From OpenAPI to MCP Servers for Tool-Augmented LLMs. arXiv [Preprint]. 2025;arXiv:2507.16044. Available from: https://arxiv.org/abs/2507.16044 Widjaja F, Chen Z, Zhou J. BioinfoMCP: A Unified Platform Enabling MCP Interfaces in Agentic Bioinformatics. arXiv [Preprint]. 2025;arXiv:2510.02139. Available from: https://arxiv.org/abs/2510.02139 Wang Z, Chang Q, Patel H, Biju S, Wu CE, Liu Q, et al. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. arXiv [Preprint]. 2025;arXiv:2508.20453. Available from: https://arxiv.org/abs/2508.20453 Tanenbaum AS, Bos H. Modern Operating Systems. 5th ed. Pearson; 2022. OpenAI. Model Context Protocol (MCP) Specification. 2024. Available from: https://github.com/modelcontextprotocol OpenAI Research. MCP Security Framework and Safe Context Exchange. arXiv preprint; 2025. Almeida R. On the Stability of Memory-Dependent Multi-Agent Systems. Future Generation Computer Systems. 2025 (In Press). Available from: https://www.sciencedirect.com/science/article/pii/S0016003224008238
道德规范 **Ethics**	Please confirm by checking the box: ☒ I confirm that I have discussed ethical issues with my supervisor.

Please discuss ethical issues with your supervisor. **Please refer to Project Student Handbook section 4.1 and Project Ethics Guidance**	Summary of ethical issues: This project does not involve any ethical issues.
中期目标 **Mid-term target.** **It must be tangible outcomes, E.g. software, hardware or simulation.** **It will be assessed at the mid-term oral.**	A working prototype demonstrating preliminary kernel-level MCP module. Deliverables include: 1. A thorough survey about the evolution from traditional process-centric OS to agent-centric AI-native OS 2. Kernel module implementing agent registration & context exchange;

Work Plan (Gantt Chart)

Fill in the sub-tasks and insert a letter X in the cells to show the extent of each task

	Nov 1-15	Nov 16-30	Dec 1-15	Dec 16-31	Jan 1-15	Jan 16-31	Feb 1-15	Feb 16-28	Mar 1-15	Mar 16-31	Apr 1-15	Apr 16-30
Task 1 Analyze the evolution of OS architectures from process-centric to agent-centric paradigms and identify the key design principles of agent-native OS.												
Review traditional OS architectures	X	X	X									
Analyse agent computing requirements, context management, and fairness	X	X	X									
Compare traditional OS abstractions vs AI-native demands			X	X	X							
Formulate design principles and theoretical framework for AI-Native OS			X	X	X							
Task 2 Study the internal structure of the MCP protocol and design how to provide its functionalities from the kernel level.												
Analyse structure, context flow, and core mechanisms of MCP.	X	X	X									
Map MCP functions to kernel system calls			X	X	X							
Define kernel-side data structures (context tables, registry).					X	X	X					
Write design specification and interface documentation.					X	X	X					
Task 3 Implement a MCP-enabled kernel module and design the exposed APIs.												
Implement a lightweight user-space daemon that loads the MCP manifest, manages tools.					X	X	X					
Build the tool adapter layer for external tools.					X	X	X					
Achieve working prototype and demo integration workflow.					X	X	X					
Build an automated test runner that compares MCP tool outputs with baseline executions to verify correctness.							X	X	X			
Task 4 Evaluate the performance and scalability of the MCP-enabled kernel model, and discuss how it achieves efficiency, fairness, and security.												
Define performance metrics and design experiments					X	X	X					
Conduct performance & scalability experiments under multi-agent workloads.							X	X	X	X		
Run static & runtime security tests							X	X	X	X		

Summarize results, visualize data, evaluate the result									X	X	X	X

Early-term progress report

北京邮电大学 本科毕业设计（论文）初期进度报告

Project Early-term Progress Report

学院 **School**	International School	专业 **Programme**	**Telecommunications Engineering with Management**		
姓 **Family name**	Li	名 **First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	班级 **Class**	2022215107
论文题目 **Project Title**	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				

1. Introduction

The integration of Large Language Models (LLMs) into operating systems has catalyzed a paradigm shift from static instruction execution to dynamic, autonomous agent behavior. However, current frameworks operate atop general-purpose operating systems that lack native abstractions for managing the lifecycle of LLM inference. Relying solely on traditional process scheduling results in uncontrolled contention for the limited context window and lacks isolation mechanisms to prevent agents from monopolizing system resources or accessing unauthorized memory regions.

This project proposes an AI-Native Operating System architecture that integrates the Model Context Protocol (MCP) [1] directly into the kernel subsystem. By elevating "Agent Context" and "Tools" to first-class kernel citizens, we aim to replace passive user-space management with active kernel-level scheduling and security enforcement, thereby bridging the gap between traditional OS resource management and autonomous agent workflows.

2. Literature Review

2.1 Limitations of User-Space Agent Frameworks

Current agent architectures, such as AutoGPT [2] and Generative Agents [3], operate as user-space processes atop general-purpose operating systems. However, these traditional OS environments lack native abstractions for LLM-centric workflows. Qin et al. [4] pointed out that existing frameworks lack a unified mechanism to manage LLM resource contention. Instead of scheduled access, agents employ an inefficient "trial-and-error" approach: they

attempt to load prompts into GPU memory until hitting hardware limits (e.g., CUDA out-of-memory exceptions), forcing expensive deallocation and multiple retry attempts that severely degrade system throughput.

Furthermore, current general-purpose OS abstractions treat Agent Context as opaque data blobs, forcing frameworks to manage state retrieval sequentially in user-space, ignoring potential data-level parallelism. Lacking native parallel primitives for "Agent Context," the system fails to leverage multi-core parallelism for concurrent reasoning tasks, forcing agents to process information strictly linearly.

2.2 The "LLM as OS" Metaphor vs. Reality

Recent works like AIOS [5] , MemGPT[6] , and Xserve [7] have popularized the concept of an "OS for LLMs." MemGPT introduces a virtual memory hierarchy to manage limited context windows, Xserve introduces a system-level abstraction and service, which utilizes a virtual capability layer with a unified API to streamline the integration of AI into applications, while AIOS proposes an LLM-centric scheduler that isolates resources and services into an AIOS kernel to improve scheduling, context management, and resource utilization efficiency. However, a critical analysis reveals fundamental architectural limitations in these systems, the MemGPT focus mainly on context memory and text generation, which is not enough for an AI agent operating system. While AIOS demonstrates improved throughput across various agent benchmarks, it operates entirely as a user-space application (running on the Python interpreter). Consequently, it lacks strict isolation: its 'Access Manager' relies on high-level logic checks, meaning a malicious agent could potentially crash the host process or bypass restrictions. Furthermore, AIOS relies on software-level context interruption rather than true hardware preemptive scheduling of GPU resources, preventing it from enforcing hard real-time guarantees for mission-critical agent responses.

Also, in the security level, the existing AI operating systems lack isolation. A malicious agent can easily crash the host Python process or access unauthorized memory, as demonstrated in recent prompt-injection attacks on agentic systems [8][9].

Some mobile operating systems have begun integrating machine learning workloads directly into the system infrastructure beyond simple application-level and configured out some solutions to tackle the above problems.

In 2024, Apple introduced a system-wide intelligence layer deeply integrated into iOS and macOS[10]. A key component of its architecture is the combination of App Intents and the Semantic Index. App Intents is a Swift-based protocol that redefines application functionalities using a standardized intent schema and exposes them to the operating system, enabling the LLM to invoke applications to fulfill user instructions. The Semantic Index organizes user data across applications to provide context-aware capabilities. Apple adopts a hybrid execution model, processing sensitive tasks on device while offloading more computationally intensive tasks to Private Cloud Compute.

Google introduced Private Compute Core to bring a new layer of privacy to the Android ecosystem. for OS-level ML features [11]. Private Compute Core is an open source, secure environment that is isolated from the rest of the operating system and apps. It ensures that sensitive data processed by ML models stays isolated from the network via strict API boundaries, communicating with the cloud only through Private Compute Services for model updates [12].

While these industrial solutions validate the necessity of OS-level AI integration, they are proprietary and siloed ecosystems. They lack a unified, open interface for third-party agents to interact with the kernel's context management. This project differs by proposing an open-source, kernel-native implementation using the standardized MCP, aiming to democratize system-level agent capabilities for the Linux ecosystem.

2.3 Kernel-Level Opportunities and the MCP Protocol

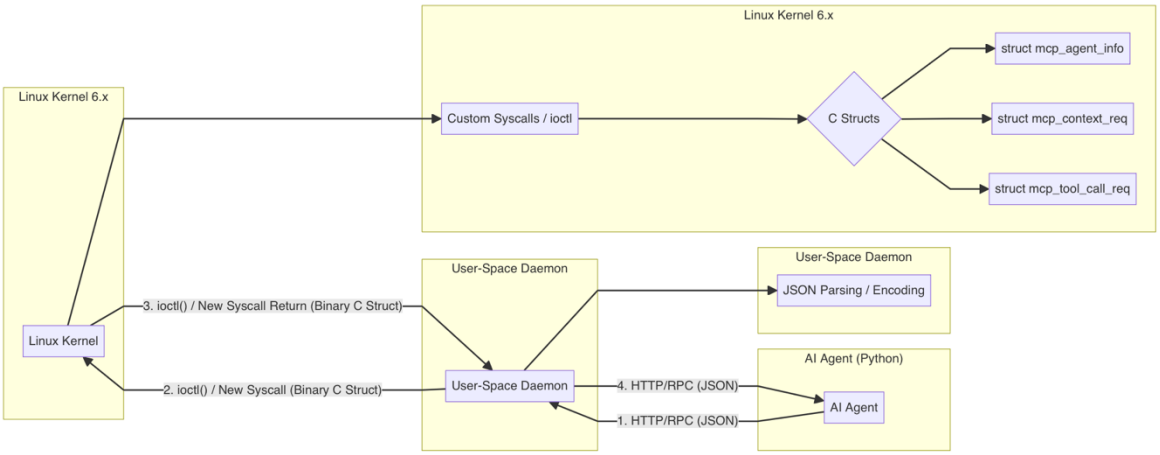
The MCP provides a standardized schema for linking agents to data, and it can effectively prevent the malicious instructions sent by agents. However, current MCP implementations are exclusively high-level (TypeScript/Python). There is a distinct lack of research on implementing MCP primitives as System Calls. This project fills this gap by exploring a Kernel-Level MCP Module to offer lower latency and mandatory access control.

3. Project Progress Summary

3.1 Completed Work

In accordance with Task 1 and the early stages of Task 2 in the project work plan, the following progress has been made:

- Theoretical Framework Analysis: I have completed a comparative analysis between traditional process-centric paradigms (Process Control Blocks - PCB) and agent-centric demands (Agent Context Blocks). I determined that the Linux task_struct needs to be extended or pointed to by a new auxiliary structure to hold MCP metadata.
- MCP Protocol Analysis: I have deeply analyzed the MCP specification and security, specifically the lifecycle of initialization and the prompts, resources, and tools primitives. I have drafted a mapping table that translates MCP JSON-RPC messages into theoretical C-language kernel structures.



- Environment Setup: A development environment has been successfully established on my MacBook using VMware, the Linux version is Ubuntu 22.04 LTS.
    - Compiled Linux Kernel 6.x source code to ensure the environment is ready for custom module insertion.
    - Installed and tested FastMCP to understand the standard user-space behavior of the MCP protocol.
    - Configured QEMU/KVM for safe kernel debugging (GDB) to prevent crashing the host machine during development.

In the future, I will deploy the system on a high-performance server to evaluate scalability and accurate performance metrics under multi-agent workloads

3.2 Comparison with Work Plan

The project is currently on schedule.

- Planned: Analyze OS architectures and MCP internal structure by mid-January.
- Actual: The literature review and architectural analysis are complete. The mapping of MCP functions to kernel system calls (Task 2) is currently in progress.
- Adjustment: The complexity of parsing JSON (used by MCP) inside the kernel was identified as a technical risk. I have decided to implement a lightweight binary serialization protocol for the kernel boundary, with a user-space daemon handling the JSON conversion, which slightly refines the approach in Task 3.

4. References

[1]. Model Context Protocol, "Model Context Protocol Specification," 2024. [Online]. Available: https://spec.modelcontextprotocol.io/

[2]. T. Richards, "Auto-GPT: An Autonomous GPT-4 Experiment," GitHub Repository, 2023. [Online]. Available: https://github.com/Significant-Gravitas/Auto-GPT

[3]. J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in Proc. ACM Symp. User Interface Softw. Technol. (UIST), 2023.

[4]. Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. ICLR, 2024.

[5]. Kai Mei et al., "AIOS: LLM Agent Operating System," arXiv preprint arXiv:2403.16971, 2024.

[6]. C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv preprint arXiv:2310.08560, 2024.

[7]. Jinrong Yang, Zimeng Wang, Rong Chen, and Haibo Chen. 2025. A System-level Abstraction and Service for Flourishing AI-powered Applications. In Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25). Association

for Computing Machinery, New York, NY, USA, 106–114. https://doi.org/10.1145/3725783.3764406

[8]. OWASP GenAI Security Project, "LLM01: Prompt Injection," OWASP Top 10 for Large Language Model Applications, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[9]. F. Liu et al., "Prompt Injection attack against LLM-integrated Applications," arXiv preprint arXiv:2306.05499, 2023.

[10]. Apple Inc., "Apple Intelligence: System Architecture and Privacy," Apple Platform Security Guide, 2024. [Online]. Available: https://security.apple.com/documentation/

[11]. Google. Android Private Compute Core: Technical Whitepaper. Google Open Source Security, 2022. [Online]. Available: https://services.google.com/fh/files/misc/android-security-paper-2024.pdf

[12]. Google. Private Compute Services. GitHub Repository, 2022. [Online]. Available: https://github.com/google/private-compute-services

是否符合进度？On schedule as per GANTT chart?
YES

下一步 Next steps:

Kernel Data Structure Definition: Define the struct mcp_agent and struct mcp_context in C code within the kernel module to store agent metadata.

Kernel Module Skeleton: Write the initial Character Device Driver (/dev/mcp_core) that allows user-space processes to "register" as agents via ioctl calls.

Prototype Integration: Develop a minimal user-space Python script using ctypes to interact with the kernel module, demonstrating basic agent registration and context retrieval.

Mid-term progress report

北京邮电大学 本科毕业设计（论文）中期进度报告

Project Mid-term Progress Report

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				
是否完成任务书中所定的中期目标？Targets met (as set in the Specification)? YES					

已完成工作 Finished work:

1. Survey Results

This study first conducts a comprehensive survey on the evolution of operating system architectures, systematically analyzing the paradigm shift from traditional process-centric models to agent-centric AI-native operating systems. Specifically, this survey identifies a structural transition across four core dimensions:

Shift in Computational Abstraction: The system's first-class citizen evolves from the "process" to the "agent," shifting the computing paradigm from deterministic execution to LLM-driven, intent-oriented generation.

Reconstruction of Resource Management: Bottlenecks transition from traditional memory allocation to LLM context windows, token consumption, and API quotas, requiring novel multi-agent scheduling algorithms.

Semanticization of IPC: Inter-module communication shifts from low-level byte streams to highly semantic interactions, utilizing JSON schemas and standardized protocols like the Model Context Protocol to route data between agents and tools.

Redefining Security Boundaries: To mitigate risks like logical privilege escalation caused by model hallucinations, system security must upgrade from hardware-based memory isolation to semantic validation and capability hash-binding.

Against this evolutionary backdrop—particularly the shift towards semantic IPC and the need for redefined security boundaries—this work further investigates the internal structure of MCP and examines how tool invocation is handled in current AIOS prototypes and multi-agent orchestration environments.

A preliminary technical analysis of these existing environments reveals three major limitations. First, tool execution is typically governed purely in user space, making it difficult to enforce strict admission control at the operating-system level. Second, while logging may exist, structured auditing and accounting are rarely maintained as kernel-backed registries with explicit counters and completion tracking. Third, fairness under multi-agent contention is rarely modeled explicitly; resource contention is usually handled heuristically with limited quantitative evaluation.

These findings directly motivated the core design principle of this project: governance must be strictly enforced by a kernel control plane, while semantic parsing and tool execution remain securely isolated in user space.

2. Requirement Analysis

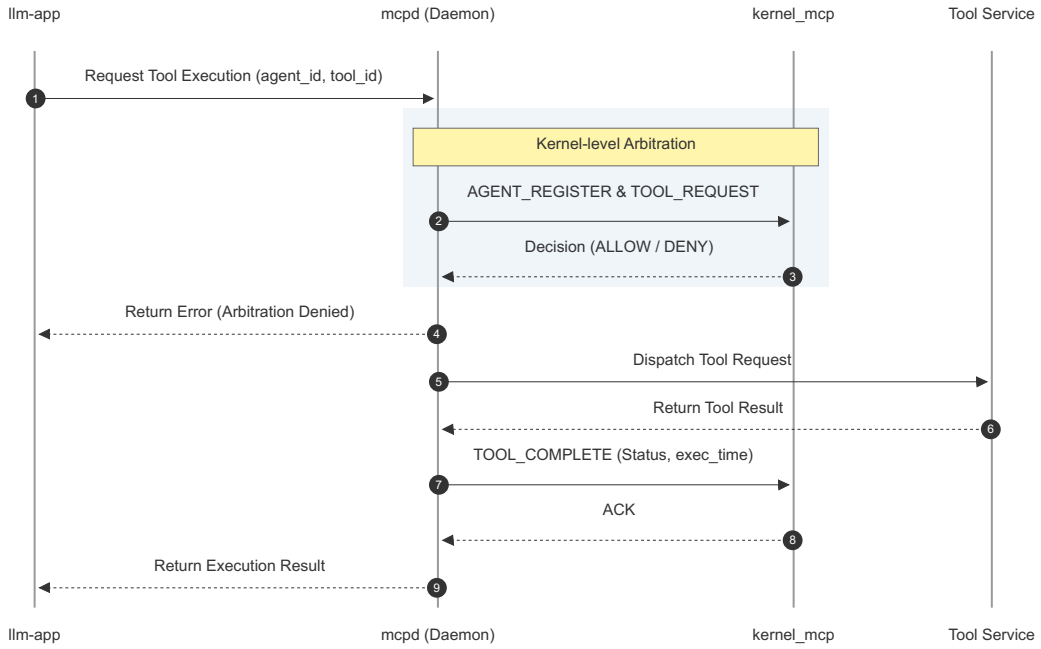
The system requirements are systematically categorized into functional requirements, architectural constraints, and non-functional requirements to ensure robust tool invocation and governance.

Functional Requirements: First, the system must provide explicit kernel-level arbitration. Every tool invocation request must yield a deterministic governance decision from the set {ALLOW, DENY, DEFER} before execution can proceed. Second, the system must support semantic discovery, enabling LLM-based clients to dynamically retrieve tool descriptions, input schemas, and execution examples for automated tool selection. Third, it must support cross-application tool invocation, requiring tools to be hosted and managed as independent resident services rather than ad-hoc inline scripts.

Architectural Constraints: To minimize kernel complexity and mitigate security risks, the system imposes a strict data-plane constraint: JSON parsing is strictly prohibited inside the kernel. The kernel boundary must only accept structured, fixed-size governance attributes (e.g., tool_id, agent_id, perm, cost, and tool_hash).

Non-Functional Requirements: From a non-functional perspective, the system must enforce strict semantic consistency between user-space application manifests and kernel-side registries to prevent semantic drift or substitution attacks. Additionally, the system must guarantee measurable fairness under concurrent multi-agent workloads, ensuring that resource contention is governed quantitatively rather than heuristically. Finally, the system must provide high runtime observability, exposing governance states, audit counters, and registry data to facilitate monitoring and debugging.

3. System Design



The implemented architecture is organized into four layers: an llm-app client layer, an mcpd gateway layer, a kernel_mcp governance layer, and a resident tool service execution layer. At runtime, the llm-app sends a framed JSON request to mcpd through a centralized Unix Domain Socket. The gateway performs routing using semantic metadata and issues a structured TOOL_REQUEST to the kernel via Generic Netlink, carrying only fixed-size governance attributes (agent_id, tool_id, perm/cost references, tool_hash, and related control fields). The kernel returns an arbitration decision (ALLOW, DENY, or DEFER). When ALLOW is returned, mcpd forwards the full JSON payload to the corresponding application-specific resident service endpoint. After

execution, mcpd reports completion back to the kernel via TOOL_COMPLETE, including status and latency, and returns the result to the client.

This design enforces a strict control-plane/data-plane separation. The kernel implements the control plane and does not parse or buffer unbounded JSON payloads. The data plane is handled entirely in user space: mcpd is the sole gateway that dispatches requests to resident tool services. Under the current threat model, clients do not directly interact with tool-service IPC endpoints; tool services accept requests routed through mcpd, which reduces the risk of bypassing kernel arbitration.

To prevent semantic drift between user-space manifests and kernel-side governance records, the system uses a canonical hash-binding mechanism. The hash is computed over stable capability attributes (tool_id, name, perm, cost, description, input_schema, and examples) while excluding runtime metadata such as endpoint paths and process identifiers. During startup, reconcile_kernel.py performs one-to-one comparison between manifest-defined tool entries and kernel registry entries; any mismatch triggers fail-fast termination. During runtime arbitration, tool_hash consistency checks provide an additional guardrail against executing a tool definition whose semantics diverge from the kernel-approved baseline.

4. Prototype Implementation

While the early-term report proposed using a character device and ioctl for kernel-user communication, the implementation transitioned to Generic Netlink. This adjustment provides a more structured and extensible IPC mechanism for multiplexing tool requests compared to legacy ioctl boundaries. A functional prototype has been implemented across kernel space and user space.

On the kernel side, the kernel_mcp module provides Generic Netlink commands for tool/agent registration, request arbitration, and completion reporting. It maintains in-kernel registries for tools and agents, implements per-agent token buckets for rate governance, and updates audit/accounting counters. These counters and registry snapshots are exposed via sysfs for observability. Basic stability checks were conducted through repeated module load/unload cycles, and no kernel warnings or crashes were observed during these tests.

On the user-space side, the mcpd gateway loads application manifests, computes canonical semantic hashes, performs startup reconciliation, enforces arbitration outcomes, forwards requests to resident tool services, and reports completion back to the kernel. For client-facing discovery, mcpd exposes semantic-only tool fields (descriptions, schemas, and examples) while avoiding leakage of runtime routing details such as service endpoints.

In the execution layer, multiple tools were migrated to persistent resident services over Unix Domain Sockets. A shared service library implements a 4-byte big-endian length-prefixed JSON protocol and supports concurrent handling via a thread-per-connection model. Each tool module supports both a debug mode and a resident service mode for development convenience. Security-oriented validations were added where applicable, including path restriction checks for file-related tools, safe evaluation constraints for expression-based tools, and basic schema/parameter validation.

These manifest snippets below illustrate how linux-mcp separates semantics from governance. The fields description, input_schema, and examples define tool semantics for discovery and selection in the llm-app layer, while runtime routing metadata (mode, endpoint, service_path, app_impl) is consumed by mcpd to dispatch requests to the correct resident tool service. Governance-related attributes form the canonical binding used for kernel-side arbitration and for startup reconciliation, ensuring a strict one-to-one correspondence between user-space manifests and the kernel registry. This design keeps JSON payload parsing and natural-language semantics in user space, while allowing the kernel control plane to enforce admission control and accounting using fixed-size structured identifiers.

```
{
  "app_id": "file_manager_app",
  "app_name": "File Manager App",
```

```
  "mode": "uds_service",
  "endpoint": "/tmp/linux-mcp-apps/file_manager_app.sock",
  "service_path": "tool-app/app_service.py",
  "app_impl": "tool-app/apps/file_manager_app.py",
  "tools": [
    {
      "tool_id": 12,
      "name": "file_delete",
      "perm": 1,
      "cost": 1,
      "handler": "file_delete",
      "description": "Delete a file (or directory with recursive=true) under repository root with safety
checks.",
      "input_schema": {
        "type": "object",
        "required": ["path"],
        "properties": {
            "path": { "type": "string" },
            "recursive": { "type": "boolean" },
            "allow_missing": { "type": "boolean" }
            },
        "additionalProperties": false
      },
      "examples": [
        { "path": "tmp/demo_created_by_mcp.txt" },
        { "payload": { "path": "tmp/old_logs", "recursive": true }}
      ]
    }
  ],
…
}
```


5. Functional Validation and System Integration

At the current stage, experimental validation focused on end-to-end system integration and functional correctness. Build validation confirmed the successful compilation of all kernel modules and C components, as well as the stable execution of the Python-based gateway and tool services. For LLM integration, the llm-app client was successfully connected to the DeepSeek large language model via API. Tests demonstrated that DeepSeek could correctly comprehend the provided tool manifests and generate well-formed JSON tool-invocation requests according to the protocol.

Crucially, the end-to-end closed-loop workflow was successfully verified across multiple different tools. The runtime observations confirmed the following sequence: user inputs trigger LLM responses; the llm-app submits tool requests to the mcpd gateway; the gateway initiates a governance check at the kernel level; the kernel successfully performs arbitration to evaluate the request; upon an ALLOW decision, the gateway dispatches the execution to the correct external tool-app; the tool executes its internal logic properly; and finally, the result is accurately propagated back to the llm-app. This functional testing demonstrates that the framework achieves its primary goal: enabling LLM-driven tool execution while enforcing strict kernel-level scrutiny.

6. Observability and GUI Integration

A graphical demo client was implemented in the llm-app layer to support interactive end-to-end validation. The GUI is built with PySide6 and connects to the gateway exclusively through the

centralized Unix Domain Socket interface exposed by mcpd. It provides an interactive workflow that mirrors the CLI pipeline: the GUI refreshes the application catalog via {"sys":"list_apps"}, retrieves tool catalogs via {"sys":"list_tools"}, and issues execution requests using {"kind":"tool:exec", ...} so that every invocation still goes through kernel arbitration and completion reporting enforced by mcpd .

The GUI exposes two practical observability surfaces during demonstration. First, it visualizes the discovered app/tool catalog after refreshing from mcpd, making it easy to confirm that the client only sees semantic fields and the tool hash, rather than runtime routing details . Second, for each user input, it shows the selected app and tool, the selector source and rationale, the request id, and the execution response including timing (t_ms), which helps trace the full loop in a single screen . The GUI reuses the same selection logic shared with the CLI, allowing consistent behavior across interactive and scripted experiments .

7. Summary

A functional linux-mcp prototype has been successfully implemented and validated, achieving kernel-governed tool invocation while strictly isolating tool semantics and execution within user space. The runtime workflow is systematically orchestrated: the llm-app (CLI/GUI) discovers tools via manifest-provided schemas and submits execution requests exclusively to the mcpd gateway. The gateway then queries the kernel via Generic Netlink to obtain an arbitration decision. Only upon receiving an ALLOW signal does mcpd dispatch the payload to per-app resident UDS services, subsequently reporting a TOOL_COMPLETE status back to the kernel for strict resource accounting. To guarantee system integrity, the design incorporates a fail-fast startup reconciliation mechanism utilizing canonical semantic hashing, which effectively prevents manifest-kernel semantic drift. Furthermore, the architecture is reinforced with kernel-side registries exposed via sysfs for observability, and persistent tool services safeguarded by length-prefixed JSON IPC and rigorous input validation.

尚需完成的任务 Work to do:

First, the arbitration model can be extended to support more granular governance, including hierarchical permission tiers, dynamic cost budgets, and adaptive backoff algorithms for deferred requests.
Second, deeper performance profiling such as syscall-level tracing is planned to rigorously quantify the IPC overhead and benchmark the architecture against traditional subprocess invocations under diverse concurrent workloads.
Finally, system observability will be further improved by expanding sysfs telemetry interfaces and developing standardized, automated test suites to facilitate reproducible evaluations in production environments.

存在问题 Problems:

There are two gaps remain before it can be considered complete and thesis-ready.

The first is security and trust-boundary clarity for high-risk tools (especially file operations). The current design correctly routes execution through a single gateway and uses manifests plus schema constraints, but the defense-grade security story still depends on clearly defined assumptions and systematic edge-case coverage. What is missing is a rigorous threat model (who is trusted, who can connect to which socket, what the attacker can do) and a set of adversarial/edge-case tests that demonstrate the boundary holds under realistic misuse scenarios.

The second is the report still needs stronger quantitative support on performance overhead and correctness under failure modes. A syscall-level comparative analysis would make the overhead of UDS framing and Netlink arbitration and dispatch concrete and comparable to baselines like subprocess invocation or direct syscall workflows.

拟采取的办法 Solutions:

First, to solidify the security and trust boundaries, a rigorous threat model and explicit trust assumptions will be formalized. At the implementation level, the system will enforce a strict, consistent sandboxing mechanism confined to the repository root. This containment strategy will be achieved by actively rejecting absolute paths and directory traversal attempts, as well as mitigating symlink escape vulnerabilities through strict realpath validation and the disabling of symlink resolutions for destructive operations. Additionally, semantic guardrails will be significantly tightened for high-risk actions such as file_delete. The robustness of these architectural boundaries will be empirically validated using targeted adversarial testing and edge-case injection scenarios to simulate realistic misuse.

Second, to strengthen the quantitative evaluation framework, the study will conduct a granular syscall-level overhead analysis utilizing system profiling utilities. This profiling will establish concrete performance metrics that will be directly compared against traditional subprocess-based and direct-invocation baselines. Furthermore, the correctness benchmarking suite will be significantly extended to cover diverse failure modes and exceptional states. This expansion includes validating the fail-fast termination logic during manifest-kernel hash mismatches, evaluating the system's backoff behavior during DEFER arbitration, and testing system resilience against IPC timeouts, partial execution failures, and service restart recovery. To ensure scientific rigor and reproducibility, all benchmarking procedures will be fully automated using standardized scripts designed for structured telemetry and metrics collection.

论文结构 Structure of the final report: (Chapter headings and section sub headings)

Chapter 1: Introduction
Chapter 2: Background
- **2.1 Traditional Operating System**
- **2.2 Existing AI Operating System**
- **2.3 Model Context Protocol**

Chapter 3: Design and Implementation
- **3.1 Overview**
- **3.2 Workflow and Protocols**
- **3.3 Kernel-Side Implementation**
- **3.4 User-Space Implementation**
- **3.5 Safety and Consistency Mechanisms**

Chapter 4: Results and Discussion
- **4.1 Experimental Setup**
- **4.2 Results**
- **4.3 Case Study**
- **4.4 Discussion**

Chapter 5: Conclusion and future work
- **5.1 Conclusion**
- **5.3 Future work**

Supervision log

北京邮电大学 本科毕业设计（论文）教师指导记录表

Project Supervision Log

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				

Please record supervision log using the format below:

Date: dd-mm-yyyy
Supervision type: face-to-face meeting/online meeting/email/messages/other (please specify)
Summary:

Date: 07-11-2025
Supervision type: online meeting
Summary: discussed the project scope and agreed on the overall direction of the linux-mcp prototype, separating kernel governance from user space tool orchestration.

Date: 05-12-2025
Supervision type: online meeting
Summary: reviewed the system architecture, including the Generic Netlink control plane in kernel-mcp and the role of mcpd as the central execution gateway.

Date: 19-12-2025
Supervision type: online meeting
Summary: discussed the interaction flow between the kernel module, mcpd, and the userspace clients, and refined the responsibilities of each layer in the system.

Date: 16-01-2026
Supervision type: online meeting
Summary: discussed the design of manifest-based tool registration and the coordination between tool-app, mcpd, and the kernel registry.

Date: 24-01-2026
Supervision type: online meeting
Summary: discussed the service structure, including resident Unix domain socket services, handler dispatching, and safe file access constraints in the file management tools.

Date: 03-03-2026
Supervision type: online meeting
Summary: discussed the organization of tools into separate apps, including calculator, utility, settings, and file manager modules, to improve maintainability and semantic clarity.

Date: 10-03-2026
Supervision type: online meeting

Summary: discussed the latest progress on semantic hash binding, tool and app discovery in llm-app, and the overall end-to-end workflow from request submission to controlled tool execution.
Date: 24-03-2026
Supervision type: online meeting
Summary: discussed the integration testing strategy across kernel-mcp, mcpd, and the userspace tool apps, and identified key failure modes to cover in the evaluation.
Date: 07-04-2026
Supervision type: online meeting
Summary: discussed the structure and chapter outline of the dissertation, including how to present the system architecture, design decisions, and experimental results.
Date: 14-04-2026
Supervision type: email
Summary: received feedback on the first draft of the introduction and background chapters, focusing on motivation framing and the positioning of MCP within the broader AI-native OS landscape.
Date: 20-04-2026
Supervision type: online meeting
Summary: reviewed the revised draft, discussed refinements to the evaluation chapter, and agreed on the remaining tasks before final submission.

Additional Appendices

A.1 Wire Protocol Reference

This section provides the complete field specifications and representative implementation fragments for the three inter-component protocols described in Section 3.4.

Complete llm-app to mcpd message reference. The `tool:exec` request carries the tool hash as a 64-character hexadecimal string. If a prior `DEFER` decision returned a `ticket_id`, the client includes it in a replay request:

Listing 1: tool:exec request and response messages (llm-app → mcpd).

```
// tool:exec request
{ "kind": "tool:exec",  "req_id": 42,
  "session_id": "sess-abc", "agent_id": "agent-1",
  "app_id": "sys_app",  "tool_id": 1,
  "tool_hash": "a3f2...c91b",        // 64-char SHA-256 hex
  "payload": {"path": "/tmp"},
  "approval_ticket_id": null }       // set on replay after DEFER

// tool:exec response (ALLOW)
{ "req_id": 42, "status": "ok",
  "result": {"os": "Linux 6.8", "cpu": "4-core"},
  "decision": "ALLOW", "reason": "admitted",
  "ticket_id": 0,  "t_ms": 3 }

// tool:exec response (DEFER)
{ "req_id": 42, "status": "error",
  "result": {}, "error": "deferred pending approval",
  "decision": "DEFER", "reason": "risk_tag:sensitive_read",
  "ticket_id": 7,  "t_ms": 1 }
```

The `approval_reply` message is issued by the client after the user confirms a deferred operation:

Listing 2: approval_reply message (llm-app → mcpd, issued after user confirms a deferred operation).

```
// approval_reply request
{ "sys": "approval_reply",  "session_id": "sess-abc",
  "ticket_id": 7,  "decision": "approve",
  "reason": "user confirmed",  "ttl_ms": 30000 }
```

mcpd to tool-app message schema. Once the kernel returns `ALLOW`, `mcpd` forwards the request to the relevant tool service over a per-application UDS connection using the minimal five-

field schema below. Tool services implement only this contract; they require no knowledge of sessions, kernel tickets, or the Netlink protocol. The `operation` field is the dispatch string a tool service routes on and is declared per-tool in the manifest; it is distinct from the `tool_id` (numeric, used by the kernel for registry indexing) and from the tool's human-readable `name`.

Table 10: `mcpd–tool-app` JSON-RPC message fields.

Field	**Dir**	**Type**	**Description**
`req_id`	req	*u64*	Per-invocation correlator
`agent_id`	req	*string*	Requesting agent identifier
`tool_id`	req	*u32*	Numeric tool identifier from the manifest
`operation`	req	*string*	Operation name within the tool
`payload`	req	*object*	Validated input matching the tool's schema
`req_id`	resp	*u64*	Must echo the request `req_id`
`status`	resp	*string*	`"ok"` or `"error"`
`result`	resp	*object*	Tool output on success; `{}` on failure
`error`	resp	*string*	Empty on success; error description on failure
`t_ms`	resp	*u32*	Tool-side execution time in milliseconds

Complete Netlink attribute encoding. All attributes are 4-byte aligned. String attributes are NUL-terminated and passed as raw bytes. The following Python fragment shows how `mcpd` encodes a `TOOL_REQUEST` message:

Listing 3: Python fragment: encoding a `TOOL_REQUEST` Generic Netlink message (mcpd → kernel).

```python
import struct, socket, hashlib

FAMILY_NAME        = b"KERNEL_MCP\x00"
CMD_TOOL_REQUEST   = 10
ATTR_AGENT_ID,   ATTR_TOOL_ID     = 4,  2
ATTR_REQ_ID,     ATTR_TOOL_HASH   = 1,  19
ATTR_AGENT_BINDING, ATTR_AGENT_EPOCH = 28, 29

def build_nla(attr_type, data):
    # NLA header: 2-byte length, 2-byte type; 4-byte aligned
    hdr = struct.pack("HH", len(data) + 4, attr_type)
    pad = (4 - len(data) % 4) % 4
    return hdr + data + b"\x00" * pad

def encode_tool_request(agent_id, tool_id, req_id,
                        tool_hash, binding, epoch):
    attrs = (
        build_nla(ATTR_AGENT_ID,       agent_id.encode() + b"\x00") +
        build_nla(ATTR_TOOL_ID,        struct.pack("=I", tool_id))   +
        build_nla(ATTR_REQ_ID,         struct.pack("=Q", req_id))    +
        build_nla(ATTR_TOOL_HASH,      tool_hash.encode() + b"\x00") +
        build_nla(ATTR_AGENT_BINDING,  struct.pack("=Q", binding))   +
        build_nla(ATTR_AGENT_EPOCH,    struct.pack("=Q", epoch))
    )
    # Generic Netlink header: cmd (1 B), version (1 B), reserved (2 B)
    genl_hdr = struct.pack("BBH", CMD_TOOL_REQUEST, 1, 0)
    return genl_hdr + attrs
```

Table 11 lists the attributes exercised on the admission-critical `TOOL_REQUEST`/`TOOL_DECISION` and approval paths. The full family also defines auxiliary attributes for registration, status reporting, payload/response audit hashing, and epoch bookkeeping; the subset reproduced inline in Section 3.4 is a further reduction of the request-critical fields shown here.

Table 11: Core KERNEL_MCP TLV attributes on the admission path. All strings are NUL-terminated and 4-byte aligned. Registration and reconciliation paths additionally carry a CATALOG_EPOCH attribute (id 34) that advances when a tool's manifest is re-registered; mcpd reads it back after each TOOL_REGISTER to compare against a session's opened_at_epoch.

Attribute	**#**	**Type**	**Description**
REQ_ID	1	*u64*	Per-invocation request identifier
TOOL_ID	2	*u32*	Numeric tool identifier
AGENT_ID	4	*string*	Agent identifier
TOOL_HASH	19	*string*	64-char hex SHA-256 of manifest fields
BINARY_HASH	33	*string*	SHA-256 of backend's /proc/<pid>/exe; composite with on-disk script bytes for interpreter-hosted tools; computed by mcpd at live probe time
DECISION	17	*u32*	Kernel verdict: 1=ALLOW, 2=DENY, 3=DEFER
TICKET_ID	22	*u64*	Approval ticket handle (DEFER path)
APPROVAL_DECISION	23	*u32*	1=approve, 2=deny, 3=revoke
AGENT_BINDING	28	*u64*	Session binding hash over SO_PEERCRED, computed by mcpd
AGENT_EPOCH	29	*u64*	Session opened-at epoch, gating cross-session replay

Semantic hash computation. The reference implementation of the semantic hash defined in Section 3.5 is reproduced below; the rationale for the field selection and the no-truncation policy is in that section and is not repeated here.

Listing 4: Python fragment: computing the manifest-derived semantic hash used for tool identity.

```python
import json, hashlib

SEMANTIC_FIELDS = [
    "tool_id", "name", "app_id", "app_name",
    "risk_tags", "description", "input_schema",
    "examples", "path_semantics", "approval_policy",
    "operation",   # in-hash: repoints the RPC method executed on the backend
]

def semantic_hash(tool_manifest: dict) -> str:
    canonical = {f: tool_manifest[f] for f in SEMANTIC_FIELDS}
    blob = json.dumps(canonical,
                      sort_keys=True,
                      separators=(",", ":"),
                      ensure_ascii=True).encode("utf-8")
    return hashlib.sha256(blob).hexdigest()  # 64-char hex; never truncated
```

Table 12: Manifest field specification. ✓ = included in the SHA-256 semantic identity hash; ✗ = excluded (operational metadata whose change should not invalidate a tool registration). Fields are grouped by level: application-level fields appear once per `app_id`; tool-level fields appear once per tool entry.

Field	In hash?	Description
Application-level fields		
`app_id`	✓	Unique application identifier string
`app_name`	✓	Human-readable application name
`transport`	✗	Must be `"uds_rpc"`
`endpoint`	✗	Absolute socket path under `/tmp/linux-mcp-apps/`
Tool-level fields		
`tool_id`	✓	Numeric identifier, unique within deployment
`name`	✓	Tool name presented to the LLM planner
`description`	✓	Natural-language description of tool behaviour
`risk_tags`	✓	Risk labels controlling kernel DEFER (e.g., `"sensitive_read"`)
`input_schema`	✓	JSON Schema object for payload validation
`examples`	✓	Example invocation payloads
`path_semantics`	✓	Path interpretation hints (e.g., `"repo_rel"`)
`approval_policy`	✓	Object specifying when user confirmation is required (e.g., `{"user_confirmation": {"when": "always"}}`); defaults to `{}` for unrestricted tools
`operation`	✓	RPC method invoked on the backend; in-hash so a retarget changes tool identity
`timeout_ms`	✗	Per-invocation execution timeout (operational metadata)

A.2 System Comparison Table

Table 13: Comparison of representative agent systems. The key differentiator for linux-mcp is the combination of cryptographic tool identity and kernel-held durable state, both absent from all user-space frameworks.

System	Control-plane placement	Tool identity	State durability	Trust anchor & security boundary
linux-mcp (this work)	Kernel module + user-space gateway	SHA-256 of JSON manifest; kernel-verified at every invocation	Approval tickets and registry survive daemon crash (kernel-held)	**Kernel**; enforcement is independent of gateway correctness
`OpenClaw` [8]	User-space gateway (persistent process)	Skill name/version string in gateway registry	Lost on process restart	User-space process; bypassable via logic flaws [26, 27]
Harness Agent [21]	Cloud-hosted pipeline orchestrator	Pipeline step + RBAC connector identity	Cloud-hosted; platform-resilient	SaaS control plane; RBAC-based governance
AIOS / AgentOS [7, 19]	New OS-level agent kernel (proposed)	Agent/tool registration in LLM scheduler	Not evaluated (proposal stage)	New OS substrate; full redesign
LangChain / LangGraph [13, 14]	In-process user-space library	Tool name string; no cryptographic binding	Lost on process crash	Application process; no inter-invocation isolation
Apple Intelligence [22]	On-device OS + Private Cloud Compute	OS capability entitlements	High; OS-persistent services	Hardware attestation + OS entitlements; closed ecosystem

A.3 Experiment Kernel Interface

The latency, stage-ablation, and extended-attack experiments in Chapter 4 rely on three kernel-side additions that live behind compile-time and module-parameter switches so that a single binary can serve both production and benchmark roles. This section documents their UAPI footprint and the peer-credential patch surfaced by the cross-UID attack rerun.

Noise-floor command: `KERNEL_MCP_CMD_NOOP`. The paired ablation anchors every stage cost against a Generic Netlink round-trip floor measured by a minimum-work handler that returns immediately after command dispatch. The command is appended to the existing `KERNEL_MCP` family at a stable numeric ID so that older `mcpd` builds continue to work unchanged:

Listing 5: UAPI additions for the noise-floor command and the stage-ablation experiment flags (kernel-mcp/include/uapi/linux/kernel_mcp_schema.h).

```
/* New command: stable numeric ID, append-only. */
#define KERNEL_MCP_CMD_NOOP                15

/* Compile-time experiment flags carried in
   KERNEL_MCP_ATTR_EXPERIMENT_FLAGS
 * on KMCP_CMD_TOOL_REQUEST. Unknown bits are ignored. */
#define KERNEL_MCP_EXPERIMENT_SKIP_LOOKUPS  (1U << 0)
#define KERNEL_MCP_EXPERIMENT_SKIP_HASH     (1U << 1)
#define KERNEL_MCP_EXPERIMENT_SKIP_BINDING  (1U << 2)
#define KERNEL_MCP_EXPERIMENT_SKIP_TICKET   (1U << 3)
```

The `SKIP_*` flags are honoured only under an experiment-enabled build (`CONFIG_KERNEL_MCP_EXPERIMENT=y`); production builds ignore the attribute entirely, so the mere presence of the UAPI constants does not weaken the enforcement path. Under the benign workload used by the ablation runner, the `SKIP_HASH` and `SKIP_BINDING` guards are statically dead code (short-circuited by a valid request before the flag is consulted) and their paired deltas correctly return zero to within noise; they are retained for methodology-consistency checks rather than as meaningful stage costs.

Peer-credential enforcement: `require_peer_cred`. The cross-UID attack rerun surfaced a defect in the stock kernel module: `kernel_mcp_cmd_tool_request` never compared the `NETLINK_CB(skb)` peer credential against the `uid` recorded at agent registration, which allowed a crafted-credential replay from a second UID to reach the tool-request handler untouched. The fix is exposed as a module parameter so that the A/B comparison in Chapter 4 could run against a single binary:

Listing 6: Peer-credential enforcement patch in `kernel_mcp_main.c`, gated behind the `require_peer_cred` module parameter.

```
static bool require_peer_cred = false;
module_param_named(require_peer_cred, require_peer_cred, bool,
    0644);
MODULE_PARM_DESC(require_peer_cred,
    "Reject TOOL_REQUEST if NETLINK_CB peer UID != agent->uid");

static int kernel_mcp_cmd_tool_request(struct sk_buff *skb,
                                       struct genl_info *info)
{
    if (require_peer_cred) {
        kuid_t peer = sock_i_uid(NETLINK_CB(skb).sk);
        if (!uid_eq(peer, agent->uid))
            return -EPERM;           /* cross-uid replay; reject
                */
    }
    /* ... existing admission pipeline ... */
}
```

The parameter defaults to `false` in the artefact published alongside this report so that the extended-attack experiment can exercise both the pre-patch and post-patch code paths without a rebuild; production deployments should set it to `true` via `/sys/module/kernel_mcp/parameters/require_peer_cred` on module load. The check reads the peer credential on the same cache line that the existing `NETLINK_CB` access already touches, so the Chapter 4 latency and throughput numbers are unchanged whether the gate is on or off.

Table 14: Experiment-only kernel switches added for the Chapter 4 rerun. All four `SKIP_*` bits are ignored when the kernel is built without `CONFIG_KERNEL_MCP_EXPERIMENT=y`; `require_peer_cred` is runtime-toggleable on any build.

Artefact	Exposed as	Purpose
`KERNEL_MCP_CMD_NOOP`	UAPI command (#15)	Generic Netlink round-trip noise-floor measurement. Returns immediately after dispatch; used by the kernel-ablation runner to anchor µs-level claims.
`SKIP_LOOKUPS`	`EXPERIMENT_FLAGS` bit 0	Bypasses the tool and agent registry walks. Used as the `skip_lookups` bar in Figure 6 and as the bypass mode in the paired registry+lookup delta in Table 7.
`SKIP_HASH`	`EXPERIMENT_FLAGS` bit 1	Bypasses the hash-mismatch guard. Guard body is dead under benign credentials; retained as a paired consistency check.
`SKIP_BINDING`	`EXPERIMENT_FLAGS` bit 2	Bypasses the binding-mismatch guard. Also dead under benign credentials; paired consistency check.
`SKIP_TICKET`	`EXPERIMENT_FLAGS` bit 3	Bypasses the approval-ticket issuance and consumption branch. Used as the `ticket_trigger_skip` mode to isolate the approval-ticket body cost.
`require_peer_cred`	Module parameter (`bool`, 0644)	Enables the `NETLINK_CB` peer-UID check in `kernel_mcp_cmd_tool_request`. Default `false` so that the cross-UID experiment can toggle it at runtime; should be set to `true` in production.

Risk and Environmental Impact Assessment

This project develops linux-mcp, a kernel-assisted control plane for AI tool invocation that splits semantic interpretation and tool execution in user space from admission control in the kernel. Because the system spans an LLM-facing application, a user-space gateway, a kernel module, and external tools, it carries technical, security, and resource-related risks. Following the prescribed method, each risk is assigned a likelihood L and a consequence C, and scored as $R = L \cdot C$. Table 15 summarises the assessment.

Technical risks. The dominant technical risk is kernel-level instability: faults in Netlink handling, synchronisation, or state management could cause crashes or inconsistent behaviour ($L = 3$, $C = 4$, $R = 12$). This is mitigated by keeping the kernel scope deliberately narrow, relying on strict interfaces, and applying staged integration tests and adversarial stress runs. A related risk is that the design fails to demonstrate a measurable advantage over hardened user-space baselines ($L = 4$, $C = 3$, $R = 12$); it is mitigated by evaluating against both conventional and hardened baselines under performance and adversarial workloads. Cross-component integration failure between `llm-app`, `mcpd`, the kernel module, and tool services ($L = 3$, $C = 2$, $R = 6$) is controlled through explicit protocols, schema synchronisation, and end-to-end validation.

Security and operational risks. The central operational risk is unsafe or unauthorized tool invocation, where a fault in the admission path allows an action that should have been denied or deferred ($L = 3$, $C = 4$, $R = 12$). Mitigations include deny-by-default admission, explicit approval binding, auditable state transitions, and attack-oriented testing. A closely related risk is overstating the security guarantees: kernel placement strengthens specific invariants but does not eliminate threats from semantic errors in user space or unsafe tool implementations ($L = 3$, $C = 3$, $R = 9$). This is addressed by stating the threat model precisely and distinguishing kernel-enforced from out-of-scope properties. Additional arbitration and approval steps may also degrade responsiveness under concurrent load ($L = 3$, $C = 2$, $R = 6$), which is mitigated by latency analysis, concurrency benchmarking, and fast-path optimisation for common low-risk calls.

Environmental and resource impact. Direct environmental impact is limited because the project is software-based, but repeated benchmarking, adversarial testing, and kernel debugging in virtual machines consume CPU time and electricity ($L = 4$, $C = 2$, $R = 8$). This is reduced through pre-planned experiment schedules, automated result collection, and avoidance of redundant reruns. The same workflow discipline also mitigates the related risk of development time lost to unstable experiments or failed integration ($L = 4$, $C = 2$, $R = 8$), together with reproducible scripts and checkpointing of experiment outputs. Finally, dependence on external tooling such as LLM APIs, proxies, or virtualization infrastructure ($L = 3$, $C = 2$, $R = 6$) is reduced by separating the core evaluation from optional cloud dependencies and by supporting local or mocked execution paths where possible.

Table 15: Summary of identified risks and mitigations for linux-mcp.

Risk	*L*	*C*	*R*	**Rating**	**Mitigation**
Kernel-level instability	3	4	12	Significant	Narrow kernel scope; strict interfaces; staged and stress testing
Weak advantage over user-space baselines	4	3	12	Significant	Strong baselines; adversarial and quantitative evaluation
Cross-component integration failure	3	2	6	Moderate	Explicit protocols; schema sync; end-to-end validation
Unsafe or unauthorized tool invocation	3	4	12	Significant	Deny-by-default; approval binding; auditability; misuse testing
Overestimated security guarantees	3	3	9	Significant	Precise threat model; clear scope of kernel-enforced properties
Added latency or reduced throughput	3	2	6	Moderate	Latency analysis; concurrency evaluation; fast-path optimisation
Additional compute and energy use	4	2	8	Significant	Efficient experiment planning; automation; fewer reruns
Lost development time and compute	4	2	8	Significant	Reproducible workflows; checkpointing; staged deployment
External dependency failure	3	2	6	Moderate	Local or mocked backends; dependency isolation